

COMPLETE TECHNICAL DOCUMENTATION

Sewage Alert Hyderabad Technical Architecture & Implementation Guide

A cloud-native, citizen-to-government civic tech platform for reporting, tracking and resolving sewage and sanitation issues — documented from the actual source code, configuration, Docker and CI/CD files in the repository.

Microservices

Spring Boot 3.5 · Java 25

Spring Cloud Gateway

Eureka

OpenFeign

JWT

RabbitMQ

MySQL 8

React 18 + TypeScript

Docker

GitHub Actions

Cloudinary

EmailJS

AWS EC2 · Vercel

Version: v1.1.0 · **Date:** August 2026 · **Scope:** Backend microservices, frontend, messaging, databases, Docker, CI/CD, cloud deployment · **Source of truth:** actual repository inspection (no invented functionality)

Table of Contents

1	Project Overview
2	Complete Application Architecture
3	End-to-End Request Flow & Sequence Diagrams
4	REST API Documentation
5	Swagger / OpenAPI
6	Spring Boot Code Architecture
7	Spring Security & JWT
8	Database Architecture
9	RabbitMQ Architecture
10	Notification & Email Flow
11	File / Image Upload Architecture
12	API Gateway & Service-to-Service Communication
13	Docker Architecture
14	CI/CD Pipeline
15	AWS / Cloud Architecture
16	Frontend Architecture
17	Configuration & Environment Management
18	Error Handling & Observability
19	Complete System Flow
20	Developer Guide
21	Troubleshooting
22	Technical Glossary
A	Appendices — Inventories & Summaries

1. Project Overview

1.1 Purpose & problem solved

Sewage Alert Hyderabad is a cloud-native, microservices-based civic tech platform that bridges the communication gap between citizens and municipal authorities. Its purpose is to give citizens a reliable, structured channel to report sewage and sanitation problems (with photo and GPS), and to give authorities a structured queue to assign, track and resolve those issues with proof of work.

The problem it solves, as documented in the repository and README:

- **No easy channel** — complaints were lost across phone calls, social media and word of mouth.
- **No transparency** — citizens could not track whether or when their issue would be fixed.
- **No accountability** — authorities lacked a structured queue, assignment and proof of work.

The answer implemented in code: one structured platform where a citizen reports with photo + GPS in under a minute, every status change is notified/stored/visible, and — as of **v1.1.0** — a complaint can only be closed as **RESOLVED** after a mandatory resolution-proof photo is uploaded (enforced at the backend, not just the UI).

1.2 User roles & responsibilities

Roles are defined in the `Role` enum in `auth-service` (`com.sewagealert.auth.model.Role`) and are stored as JWT claims and DB enum values:

Role	Responsibilities (as implemented)
CITIZEN	Register, verify email, log in, report complaints with photo + GPS, track own complaints, receive notifications, register for community events, read articles/NGO/lake/STP/pipeline pages.
AUTHORITY	Municipal staff; updates complaint status/priority via <code>PATCH /api/v1/complaints/{id}/status</code> (service re-verifies against AUTH-SERVICE).
ADMIN	Command centre: assign complaints to field officers, resolve with mandatory proof photo, manage analytics/heatmap, manage community content (events, articles, NGOs, lakes, STPs, pipelines), settings.
FIELD_OFFICER	Field staff; sees only complaints assigned to them, updates their status, resolves them with a mandatory proof photo. Ownership is enforced server-side.

Seeded accounts: the Auth Service seeds one ADMIN and three FIELD_OFFICER accounts at startup from `app.admin` / `app.field-officers.accounts` properties (see §17). Seeded staff accounts are created pre-verified.

1.3 Major application features (implemented)

- **Registration with OTP email verification** — 6-digit code, SHA-256 hashed at rest, TTL 30 min, brute-force throttling (5 attempts / 60 s lockout), resend throttled to 1/min.
- **Login with stateless JWT** (HS256, 24 h expiry; claims: sub/email/role/iat/exp).
- **Complaint lifecycle** — create with photo(s) + GPS, list/filter, status & priority updates, assignment to field officers, history/audit trail, delete.
- **Mandatory resolution-proof photo (v1.1.0)** — `POST .../resolve` with a `proofImage` file part; upload happens *before* the status change; JSON PATCH to RESOLVED is rejected.
- **Cloudinary image storage** — only URLs persisted, never image bytes.
- **Event-driven notifications** — RabbitMQ topic exchange → in-app notifications + EmailJS transactional emails.
- **Community content** — awareness events (with registrations/capacity), educational articles, NGOs, lakes, sewage pipelines, treatment plants.
- **External data integrations** — GNews (articles), Google Places (NGO search), OpenStreetMap Overpass (lakes), Telangana Government ArcGIS (STP locations).
- **Admin analytics & hotspot heatmap** (frontend: Recharts + Leaflet).
- **Role-based frontend routing** with route guards (public / citizen / admin / officer).
- **Full containerised deployment** — Docker Compose (dev) + production Compose pulling images from Docker Hub; GitHub Actions CI + image build/push.

1.4 Technology stack and rationale (from the repository)

Layer	Technology	Why it is used (as evidenced in code/config)
Runtime	Java 25, Spring Boot 3.5.5	Parent POM (<code>spring-boot-starter-parent 3.5.5</code>), <code>java.version=25</code> ; builds run on JDK 25 in CI and Docker images.
Architecture	Spring Cloud 2025.0.0 microservices	Root POM declares 7 Maven modules: <code>eureka-server</code> , <code>api-gateway</code> , <code>auth-service</code> , <code>user-service</code> , <code>complaint-service</code> , <code>community-service</code> , <code>notification-service</code> .
Routing	Spring Cloud Gateway	Single entry point on :8080, routes <code>/api/v1/**</code> to <code>lb://SERVICE-NAME</code> via Eureka.
Discovery	Netflix Eureka	Service registry on :8761; all services register and resolve peers by name.

Inter-service	OpenFeign	Declarative clients: Auth→User, Complaint→Auth/User, Notification→User (all through Eureka).
Auth	Spring Security + JWT 0.12.6 + BCrypt	Stateless JWT; BCryptPasswordEncoder; JwtAuthenticationFilter.
Persistence	Spring Data JPA / Hibernate + MySQL 8 / H2	ddl-auto: update; per-service schemas; H2 in-memory as dev default for some services.
Messaging	RabbitMQ 3.13 (Spring AMQP)	Topic exchange notification.exchange, DLQ, retries; JSON via Jackson2JsonMessageConverter.
Image storage	Cloudinary	Complaint images and resolution-proof photos; only URLs stored in MySQL.
Email	EmailJS REST API	Transactional verification/welcome mail sent server-side by Notification Service.
Frontend	React 18 + TypeScript, Vite, Tailwind, React Router 6, Recharts, Leaflet, Lucide, Framer Motion	frontend/package.json dependencies.
Containerisation	Docker + Docker Compose	Root + per-service Dockerfiles; docker-compose.yml (9 containers) and docker-compose.prod.yml (pull-only).
CI/CD	GitHub Actions	backend-ci.yml, frontend-ci.yml, docker-build-push.yml, qodana_code_quality.yml.
Hosting	AWS EC2 (backend), Vercel (frontend), Docker Hub (registry)	docker-compose.prod.yml pulls devsiddhartha21/* images; frontend/vercel.json rewrites /api to the EC2 gateway.

1.5 High-level system architecture

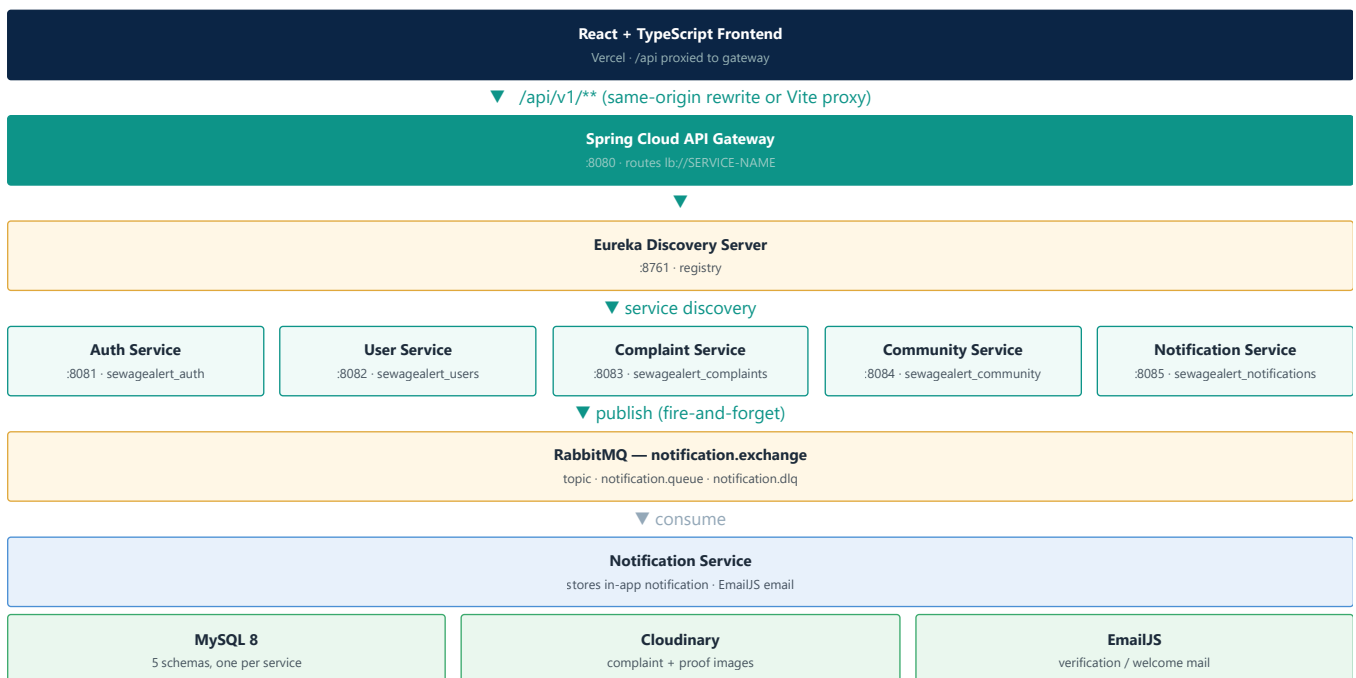


Figure 1 — High-level system architecture (all components verified in the repository)

1.6 Monolithic vs microservices

The project is a genuine **microservices** architecture, not a modular monolith. Evidence from the repository:

- **Seven separate deployable units** — each Maven module builds its own Spring Boot jar (per-service `Dockerfile` runs `mvn -pl <service> -am clean package`), and the CI workflow pushes **seven separate Docker images** to Docker Hub.
- **One database per service** — MySQL init script creates `sewagealert_auth`, `sewagealert_users`, `sewagealert_complaints`, `sewagealert_community`, `sewagealert_notifications`. No service shares another service's schema.
- **Loose coupling by convention** — cross-service references (e.g. `Complaint.createdBy`, `Notification.userId`, `UserProfile.authUserId`) are plain Long ids, *not* JPA relationships. Services resolve identity/roles via OpenFeign calls.
- **Independent ports** (8080–8085, 8761) and independent lifecycle (Eureka registration, Docker health checks, restart policies).
- **Async decoupling** — the notification path uses RabbitMQ so producers never block on email/in-app delivery.

1.7 Frontend / backend / database / messaging / infrastructure / integrations

- **Frontend** — React 18 + TypeScript SPA (see §16), talking only to the gateway over `/api/v1/**`.
- **Backend** — seven Spring Boot services (see §2).
- **Databases** — MySQL 8.4 in containers (H2 in-memory for local dev in auth/user/community services; complaint/notification default to local MySQL). See §8.
- **Messaging** — RabbitMQ 3.13-management (see §9).

- **Infrastructure** — Docker Compose bridge network; AWS EC2 backend host; Vercel frontend with SPA fallback; GitHub Actions CI/CD (see §13–15).
- **External integrations** — Cloudinary, EmailJS, GNews, Google Places, Overpass, Telangana ArcGIS (see §2.8, §11).

2. Complete Application Architecture

This section documents every service/module actually present in the repository, its package structure, responsibilities, and how it communicates.

2.1 Eureka Server (`eureka-server`)

- **Responsibility:** service discovery registry. All services register and the gateway + Feign clients resolve `1b://SERVICE-NAME` through it.
- **Key class:** `com.sewagealert.eureka.EurekaServerApplication` — `@EnableEurekaServer`.
- **Config:** port `8761`; `register-with-eureka: false`, `fetch-registry: false`; `enable-self-preservation: false`.
- **Database:** none. **APIs exposed:** Eureka dashboard/API at `/eureka/**`.

2.2 API Gateway (`api-gateway`)

- **Responsibility:** single entry point; routes `/api/v1/**` to backend services via Eureka; global CORS; raised codec buffer for multipart uploads (40 MB).
- **Key classes:** `ApiGatewayApplication` (`@EnableDiscoveryClient`), `config/OpenApiConfig` (gateway Swagger + bearerAuth scheme).
- **Config:** port `8080`; routes declared in `application.yml` — see §12 for the full route table.
- **Database:** none.

2.3 Auth Service (`auth-service`) — port 8081

Responsibility: registration (with OTP email verification), login/JWT issuance, profile retrieval, role/identity lookups for other services, admin field-officer listing, seeding of default admin/field-officer accounts.

Layer	Classes (package <code>com.sewagealert.auth</code>)
Controllers	<code>controller/AuthController</code> (public + protected API), <code>controller/AuthInternalController</code> (service-to-service role lookup)
DTOs	<code>dto/RegisterRequest</code> , <code>LoginRequest</code> , <code>VerifyCodeRequest</code> , <code>ResendVerificationRequest</code> , <code>AuthResponse</code> , <code>FieldOfficerResponse</code> , <code>UserRoleResponse</code> , <code>ApiResponse</code> , <code>CreateUserProfileRequest</code> , <code>NotificationEvent</code>
Services	<code>service/AuthService</code> , <code>service/EmailVerificationService</code> ; impl: <code>AuthServiceImpl</code> , <code>EmailVerificationServiceImpl</code>
Repositories	<code>repository/UserRepository</code> , <code>repository/EmailVerificationTokenRepository</code>
Entities	<code>model/User</code> , <code>model/EmailVerificationToken</code> , enums <code>Role</code>
Security	<code>security/SecurityConfig</code> , <code>JwtAuthenticationFilter</code> , <code>JwtTokenProvider</code>
Exception handling	<code>exception/GlobalExceptionHandler</code> + 7 custom exceptions
Producer	<code>producer/NotificationEventProducer</code> → <code>impl/NotificationEventProducerImpl</code> (publishes <code>notification.user.*</code> events)
Config	<code>config/RabbitMqConfig</code> , <code>RabbitMqProperties</code> , <code>VerificationProperties</code> , <code>AdminProperties</code> , <code>AdminSeeder</code> , <code>FieldOfficerProperties</code> , <code>FieldOfficerSeeder</code> , <code>OpenApiConfig</code>
Clients (Feign)	<code>client/UserServiceClient</code> → <code>USER-SERVICE POST /api/v1/users/internal/profile</code>
Utilities	<code>util/VerificationTokenGenerator</code> (SecureRandom OTP + SHA-256 hash)

- **Database:** `sewagealert_auth` (tables `users`, `email_verification_tokens`). Default profile: H2 in-memory; `application-mysql.yml` profile for MySQL.
- **Communication:** publishes to RabbitMQ (`notification.user.registered`, `notification.user.verified`); calls `USER-SERVICE` over Feign to create the citizen profile at registration.

2.4 User Service (`user-service`) — port 8082

Responsibility: citizen profile CRUD (name, phone, profile picture URL, address, preferences).

Layer	Classes (package <code>com.sewagealert.user</code>)
Controllers	<code>controller/UserController</code> (public-profile API), <code>controller/InternalUserController</code> (service-to-service profile lookup)
DTOs	<code>dto/ApiResponse</code> , <code>UserProfileRequest</code> , <code>UserProfileResponse</code> , <code>CreateUserProfileRequest</code>
Service	<code>service/UserService</code> → <code>impl/UserServiceImpl</code>
Repository	<code>repository/UserProfileRepository</code>

Entity	model/UserProfile (table user_profiles, auth_user_id unique)
Exception handling	exception/GlobalExceptionHandler, UserProfileNotFoundException
Config	config/OpenApiConfig

- **Database:** sewagealert_users.
- **Communication:** consumed by AUTH-SERVICE (profile creation) and COMPLAINT-SERVICE (profile validation) over Feign using internal endpoints (/api/v1/internal/users/**), which the gateway deliberately does not route.

2.5 Complaint Service (complaint-service) — port 8083

Responsibility: complaint lifecycle — creation with images + GPS, listing/filtering, status & priority updates, admin assignment, resolution with mandatory proof photo (v1.1.0), history/audit, deletion; Cloudinary image storage.

Layer	Classes (package <code>com.sewagealert.complaint</code>)
Controllers	controller/ComplaintController, AdminComplaintController (assign/resolve), FieldOfficerComplaintController (own queue + resolve)
DTOs	dto/ApiResponse, ComplaintRequest, ComplaintResponse, ComplaintStatusRequest, AssignComplaintRequest, NotificationEvent, UserProfileResponse, UserRoleResponse
Service	service/ComplaintService → impl/ComplaintServiceImpl
Repositories	repository/ComplaintRepository, ComplaintImageRepository, ComplaintHistoryRepository
Entities	model/Complaint, ComplaintImage, ComplaintHistory, enums ComplaintStatus, ComplaintPriority
Storage	storage/ImageStorageService (interface), storage/CloudinaryStorageService
Exception handling	exception/GlobalExceptionHandler + 9 custom exceptions (incl. ResolutionProofRequiredException)
Producer	producer/NotificationEventProducer → impl/... (publishes notification.created/assigned/status.updated/resolved/rejected/reopened)
Config	config/CloudinaryConfig, RabbitMqConfig, RabbitMqProperties, UploadProperties, OpenApiConfig
Clients (Feign)	client/AuthServiceClient (role verification), client/UserServiceClient (profile validation)

- **Database:** sewagealert_complaints (tables complaints, complaint_images, complaint_history).
- **Communication:** OpenFeign → AUTH-SERVICE (role), USER-SERVICE (profile); RabbitMQ producer for notifications.

2.6 Community Service (community-service) — port 8084

Responsibility: awareness events (with registrations), educational articles, NGOs, lakes, sewage pipelines, treatment plants; plus external open-data integrations (GNews, Google Places, Overpass, Telangana ArcGIS).

Layer	Classes (package <code>com.sewagealert.community</code>)
Controllers	controller/EventController, ArticleController, NgoController, LakeController, TreatmentPlantController, PipelineController
DTOs	dto/ApiResponse, EventRequest/Response, ArticleRequest/Response, NgoRequest/Response, LakeRequest/Response, TreatmentPlantRequest/Response, PipelineRequest/Response, plus external DTO packages (news, places, overpass, telangana)
Services	7 domain services + 4 external services (each with impl): EventService, ArticleService, ArticleExternalService, NgoService, NgoExternalService, LakeService, LakeExternalService, TreatmentPlantService, TreatmentPlantExternalService, PipelineService
Repositories	EventRepository, EventRegistrationRepository, ArticleRepository, NgoRepository, LakeRepository, TreatmentPlantRepository, PipelineRepository
Entities	model/Event, EventRegistration, Article, Ngo, Lake, TreatmentPlant, Pipeline
Clients (Feign, external)	client/NewsApiClient (GNews), GooglePlacesClient, OverpassClient, TelanganaStpClient (ArcGIS)
Exception handling	exception/GlobalExceptionHandler + ExternalApiException, GooglePlacesException, NewsApiException, LocationServiceException
Config	config/EventSeeder, NewsApiProperties, GooglePlacesProperties, OverpassProperties, TelanganaArcGisProperties, OpenApiConfig

- **Database:** sewagealert_community. Default profile: H2; MySQL via application-mysql.yml.
- **Communication:** external REST APIs only (Feign with configured base URLs); no RabbitMQ producer currently wired.

2.7 Notification Service (`notification-service`) — port 8085

Responsibility: consumes domain events from RabbitMQ, persists in-app notifications, serves them via REST (paginated, read/unread, soft delete), and sends transactional email via EmailJS.

Layer	Classes (package <code>com.sewagealert.notification</code>)
Consumer	<code>consumer/NotificationEventConsumer</code> (main queue), <code>consumer/DeadLetterQueueConsumer</code> (DLQ observer)
Controller	<code>controller/NotificationController</code>
DTOs	<code>dto/NotificationEvent</code> , <code>NotificationResponse</code> , <code>PagedResponse</code> , <code>ApiResponse</code>
Services	<code>service/NotificationService</code> → <code>impl/NotificationServiceImpl</code> ; <code>service/EmailNotificationService</code> → <code>impl/EmailNotificationServiceImpl</code>
Repository	<code>repository/NotificationRepository</code>
Entity	<code>model/Notification</code> ; enum <code>NotificationType</code>
Mapper / util	<code>mapper/NotificationMapper</code> , <code>util/EventTypeResolver</code> , <code>util/JsonUtils</code>
Producer	<code>producer/NotificationEventProducer</code> (re-publishes stored notifications to the delivery exchange)
Client	<code>client/EmailJsClient</code> (REST to EmailJS), <code>client/UserServiceClient</code> (Feign → USER-SERVICE)
Exception handling	<code>exception/GlobalExceptionHandler</code> + <code>NotificationNotFoundException</code> , <code>NotificationProcessingException</code> , <code>RabbitMqConsumerException</code> , <code>RabbitMqPublishException</code>
Config	<code>config/RabbitMqConfig</code> , <code>RabbitMqProperties</code> , <code>EmailJsProperties</code> , <code>AppProperties</code> , <code>OpenApiConfig</code>

- **Database:** `sewagealert_notifications` (table `notifications` with 4 indexes).
- **Communication:** RabbitMQ consumer (main queue + DLQ); EmailJS REST; optional Feign to USER-SERVICE.

2.8 External dependencies (integration summary)

External system	Consumed by	Purpose	Credential(s)
Cloudinary	<code>complaint-service</code>	Complaint images + resolution-proof photos	<code>CLOUDINARY_CLOUD_NAME/API_KEY/API_SECRET</code>
EmailJS	<code>notification-service</code>	Verification + welcome transactional email	<code>EMAILJS_*</code> (5 vars)
GNews	<code>community-service</code>	Fresh news articles (<code>/api/v4/search</code>)	<code>GNEWS_API_KEY</code>
Google Places	<code>community-service</code>	NGO search (textsearch + place details)	<code>GOOGLE_PLACES_API_KEY</code>
OpenStreetMap Overpass	<code>community-service</code>	Lake geo-data for Hyderabad bbox	key-less public API
Telangana ArcGIS (TGRAC)	<code>community-service</code>	STP locations (MapServer/7)	key-less public open data

2.9 How services communicate — and why

Mechanism	Used between	Why
Synchronous REST over Feign (Eureka lb://)	Auth→User; Complaint→Auth (role), Complaint→User (profile); Notification→User	Request/response semantics where the caller needs an immediate answer (e.g. "does this profile exist?", "what role does this user have?"). OpenFeign gives typed clients; Eureka removes hardcoded URLs.
Async RabbitMQ (fire-and-forget)	Auth/Complaint → Notification	Notifications/emails must not block the primary request. Publishing never throws — a broker outage does not break registration or complaint creation.
Direct DB	Each service → own schema	No cross-service DB access; data ownership enforced by schema boundaries.

3. End-to-End Request Flow & Sequence Diagrams

Every request follows the same spine:

```
User → Frontend (React) → API Gateway (:8080) → Service Controller → ServiceImpl → Repository → Database → Response (back out the same path)
```

Where the flow touches messaging:

```
Service (producer) → RabbitMQ notification.exchange → notification.queue → NotificationService consumer → in-app notification (DB) + EmailJS (email) → User
```

3.1 Standard authenticated request path

The frontend attaches two headers to every authenticated call (see `frontend/src/lib/api.ts`): `Authorization: Bearer <jwt>` and `X-Auth-User-Id: <id>` (from the stored session). The gateway routes by path; each service then enforces identity/roles server-side (JWT filter in auth-service; role re-verification via Feign in complaint-service).

	React Frontend	API Gateway :8080	Service Controller	ServiceImpl	Repository / DB
1	<code>fetch('/api/v1/...', headers {Authorization, X-Auth-User-Id})</code>	<code>route lb:/SERVICE</code>			
2		<code>match Path=/api/v1/**</code>	<code>@RequestHeader X-Auth-User-Id</code>		
3			<code>@Valid DTO (Bean Validation)</code>	business logic	
4				role/user checks via Feign (AUTH/USER-SERVICE)	
5				save/find via repository	JPA query → SQL
6	<code>render + toast</code>	200 OK	<code>ApiResponse.success(...)</code>	entity → DTO (fromEntity)	

3.2 User registration (+ OTP email verification)

	Citizen	Auth Service	User Service	RabbitMQ	Notificat
1	<code>POST /api/v1/auth/register {name,email,password,phone}</code>				
2		<code>existsByEmail? → EmailAlreadyExists 409</code>			
3		<code>new User(emailVerified=false) · BCrypt hash · save</code>			
4		<code>Feign POST /api/v1/users/internal/profile</code>	createProfile (best-effort)		
5		<code>createVerification → 6-digit OTP (SHA-256 hash stored, TTL 30 min)</code>			
6		<code>publish USER_REGISTERED (metadata: name,email,verificationCode)</code>		<code>notification.exchange → queue</code>	
7		<code>201 AuthResponse (NO token)</code>			<code>consume → store noti</code>
8	<code>user types the 6-digit code → POST /api/v1/auth/verify-code</code>	<code>validate hash, expiry, throttling (5/60 s)</code>			
9		<code>emailVerified=true · publish EMAIL_VERIFIED</code>		<code>→ queue</code>	<code>store + welcome ema</code>
10	<code>login allowed</code>				

Notable implementation details (from `AuthServiceImpl` / `EmailVerificationServiceImpl`): the raw OTP is never logged or persisted — only its SHA-256 digest; profile creation is best-effort (logged, account remains usable on failure); publishing is fire-and-forget (broker outage never blocks registration); the verify response is deliberately generic to prevent account enumeration.

3.3 Login / JWT generation

	Citizen	Auth Service	DB (users)	JwtTokenProvider
1	<code>POST /api/v1/auth/login {email,password}</code>			
2		<code>findByEmail → InvalidCredentials 401 if absent</code>	<code>SELECT</code>	

3		passwordEncoder.matches? → 401 on mismatch		
4		emailVerified? → 401 code EMAIL_NOT_VERIFIED		
5		generateToken(userId,email,role)		Jwts.builder().subject(id).claim(email).claim(role).issuedAt().expiration(24h).signWith(HS256,secretKey)
6	stores JWT in localStorage (sa_token)	200 AuthResponse(token,id,name,email,role)		

3.4 JWT validation on subsequent requests

	Frontend	Auth Service SecurityFilterChain	JwtAuthenticationFilter	JwtTokenProvider	
1	Authorization: Bearer <jwt>				
2		stateless · CSRF off · permitAll? → skip			
3			extractTokenFromRequest	validateToken (parseSignedClaims, verifyWith key)	
4			build UsernamePasswordAuthenticationToken(userId, null, [ROLE_role]) → SecurityContextHolder	getUserIdFromToken / getRoleFromToken	
5					Authenticated
6			invalid/expired → exception logged → 401 → frontend clears session (sa:auth-expired)		

3.5 Creating a complaint (multipart, with images)

	Citizen	Complaint Service	User Service	Cloudinary	
1	POST /api/v1/complaints (multipart: fields + images parts) + headers				
2		validateUserExists(authUserId) via Feign	GET /api/v1/internal/users/{id}		
3		validate each image (type JPG/PNG/WEBP, ≤10 MB)			
4		upload files → URLs		folder "complaints" → secure_url	
5		tx sync: on rollback → delete uploaded			
6		save Complaint(PENDING) + history + images			INSERT Complaint
7		publish COMPLAINT_CREATED			
8	201 ComplaintResponse(id,status:PENDING,imageUrls,history)				

3.6 Viewing complaints / complaint detail

- GET /api/v1/complaints — ComplaintServiceImpl.getAllComplaints() (findAll()) or getComplaintsByUser(userId) (findByCreatedBy) when the userId query param is present.
- GET /api/v1/complaints/{id} — findById → ComplaintResponse.fromEntity (flattens images to URL list + history to HistoryEntry DTOs). 404 via ComplaintNotFoundException.
- GET /api/v1/complaints/field-officer — role verified against AUTH-SERVICE, then findByAssignedTo(officerUserId).

3.7 Admin complaint assignment

	Admin	AdminComplaintController	ComplaintServiceImpl	Auth Service	DB	RabbitMQ
1	PUT /api/v1/complaints/admin/{id}/assign {fieldOfficerId} + headers					
2		X-Auth-User-Id → adminUserId	verifyRole(adminUserId, "ADMIN") via Feign	GET /api/v1/internal/auth/users/{id}/role		
3			complaint exists? terminal states (RESOLVED/REJECTED) immutable			
4			fetch officer role → must be FIELD_OFFICER	role lookup		
5			set assignedTo + history entry		UPDATE	

6			publish COMPLAINT_ASSIGNED (userId = officer)		notific
---	--	--	---	--	---------

3.8 Complaint status update (authority / officer)

- PATCH `/api/v1/complaints/{id}/status` (authority) and PATCH `/api/v1/complaints/field-officer/{id}/status` (officer, ownership enforced) both reach `applyStatusChange()`.
- `applyStatusChange` **rejects RESOLVED** with `ResolutionProofRequiredException` (400) — resolution requires the multipart resolve endpoint with a proof photo. REJECTED stores remarks; priority may be set; a history entry is always appended; a status-change event is published (RESOLVED/REJECTED/REOPENED/ASSIGNED variants chosen by the producer).

3.9 Complaint resolution with mandatory proof photo (v1.1.0)

	Admin / Officer	resolve endpoint (admin or officer)	ComplaintServiceImpl.applyResolution
1	POST <code>.../resolve</code> (multipart: proofImage + remarks/priority) + headers		
2		verifyRole (ADMIN / FIELD_OFFICER + ownership)	
3			proofImage null/empty → ResolutionProofRequiredException (neve
4			validate (JPG/PNG/WEBP ≤10 MB)
5			upload proof FIRST → proofUrl
6			tx sync: rollback → delete proof
7			status=RESOLVED, resolutionProofImageUrl, remarks, history
8			publish COMPLAINT_RESOLVED
9	ComplaintResponse{status:RESOLVED, resolutionProofImageUrl}		

Backend-enforced rule: the JSON PATCH status endpoint rejects RESOLVED; only the multipart `resolve` endpoints can close a complaint, and they require the proof image. A failed upload leaves the complaint untouched — never silently resolved.

3.10 Notification generation (async)

	Producer Service	RabbitMQ	Not
1	NotificationEventProducerImpl builds event (UUID eventId, eventType, userId, title, message, metadata)		
2	convertAndSend(exchange, routingKey, event) — fire-and-forget	topic match notification.# → notification.queue	
3			@Rabi
4			
5			
6			
7			
8		failure after retries → notification.dlx → notification.dlq (DLQ consumer logs)	

3.11 Email notification (EmailJS)

```
Business event → producer → RabbitMQ → NotificationService consumer → EmailNotificationServiceImpl
→ EmailJsClient.send(templateId, templateParams) → POST https://api.emailjs.com/api/v1.0/email/send → User inbox
```

- **Verification email** (USER_REGISTERED / EMAIL_VERIFICATION_REQUESTED): params `name`, `email`, `verification_code` (rendered as `{{verification_code}}` in the EmailJS template).
- **Welcome email** (EMAIL_VERIFIED): params `name`, `email`, `login_url` (= `FRONTEND_URL` + `/login`), only when `EMAILJS_WELCOME_TEMPLATE_ID` is configured.
- **Failure model:** send errors are logged (with eventId) and never rethrown — the stored notification and RabbitMQ ack are preserved; customers can always use resend-verification.
- **Timeouts:** connect 5 s, read 15 s (SimpleClientHttpRequestFactory) so a hung provider never stalls the consumer thread.

3.12 Other flows present in the code

- **Email verification resend:** `POST /api/v1/auth/resend-verification` — throttled 1/min per account (in-memory ConcurrentHashMap), generic response, re-publishes EMAIL_VERIFICATION_REQUESTED only if the account exists and is unverified.
- **Profile refresh:** `GET /api/v1/auth/profile` — returns a fresh JWT + user summary (frontend calls it on boot to restore session).
- **Notification feed:** `GET /api/v1/notifications?page&size` (newest first, size clamped 1–100), `GET /unread-count`, `PATCH /{id}/read`, `PATCH /read-all`, `DELETE /{id}` (soft delete). All scoped to X-Auth-User-Id.
- **Community flows:** event CRUD + `POST /{id}/register` (duplicate + capacity checks); article CRUD + `GET /latest` (GNews); NGO CRUD + `GET /search` (Google Places, Overpass fallback); lake / treatment-plant / pipeline CRUD + external fetch endpoints.
- **User profile flow:** `POST /api/v1/users` (X-Auth-User-Id), `GET /{id}`, `GET /auth/{authUserId}`, `PUT /{id}`, `DELETE /{id}`.

4. REST API Documentation

All public endpoints are served through the API Gateway at `http://<host>:8080` and share the `/api/v1` prefix. Every response is wrapped in a uniform envelope `ApiResponse<T>`:

```
{
  "success": true,
  "message": "Complaint created successfully",
  "data": { ... }      // or null for void operations
}
```

Errors use the same envelope with `success: false`, a message, and a structured `errors` map for field-validation failures (see §18).

4.1 Common conventions

Item	Convention
Base URL	<code>http://localhost:8080</code> (gateway) — services also listen directly on their own ports (8081–8085) for local debugging
Auth headers	<code>Authorization: Bearer <JWT></code> (used/validated by auth-service) and <code>X-Auth-User-Id: <user id></code> (used by all other services to identify the caller)
Content types	<code>application/json</code> for JSON bodies; <code>multipart/form-data</code> for complaint creation and resolution endpoints
Common status codes	200 OK · 201 Created · 400 Bad Request (validation) · 401 Unauthorized · 403 Forbidden · 404 Not Found · 409 Conflict · 500 Internal Server Error
Pagination	Notification feed uses <code>page</code> and <code>size</code> query params (0-based, size clamped 1–100)

4.2 Auth Service — auth-service (port 8081)

POST /api/v1/auth/register — register a citizen account

Auth	None (public)
Body	<code>{ "name": "...", "email": "...", "password": "...", "phone": 9876543210, "role": "CITIZEN" }</code> — name 2–100 chars, valid email, password 6–100 chars, phone optional, role optional (defaults to CITIZEN)
Success	201 Created — <code>ApiResponse</code> <i>without</i> token; user must verify email first
Errors	409 EmailAlreadyExists · 400 validation

POST /api/v1/auth/login — authenticate and receive JWT

Auth	None (public)
Body	<code>{ "email": "...", "password": "..." }</code>
Success	200 — <code>{ "token": "<jwt>", "type": "Bearer", "id": 1, "name": "...", "email": "...", "role": "CITIZEN" }</code>
Errors	401 InvalidCredentials · 401 EMAIL_NOT_VERIFIED (message/error code)

POST /api/v1/auth/verify-code — verify 6-digit email OTP

Auth	None (public)
Body	<code>{ "email": "...", "code": "123456" }</code>
Success	200 — generic success message (anti-enumeration)
Errors	400 invalid/expired code (generic message) · 429 too many attempts (5 per 60 s)

POST /api/v1/auth/resend-verification — resend OTP email

Auth	None (public) — requires <code>email</code> body field
Success	200 generic message
Errors	429 throttled (1 per 60 s per account)

GET /api/v1/auth/profile — refresh session / get current user

Auth	JWT (Bearer)
Success	200 — fresh <code>ApiResponse</code> (token + id, name, email, role)

Errors	401 invalid/expired token
--------	---------------------------

GET /api/v1/auth/admin/field-officers — list field officer accounts

Auth	JWT (Bearer) + ADMIN role
Success	200 — List<FieldOfficerResponse> (id, name, email)
Errors	403 if caller is not ADMIN

4.3 User Service — user-service (port 8082)

Method / Path	Purpose	Auth / Role	Notes
POST /api/v1/users	Create user profile	X-Auth-User-Id	Body: name, phone, profilePictureUrl, address, preferences
POST /api/v1/users/internal/profile	Create profile for a newly registered user	Internal (Feign from auth-service)	Creates profile when none exists
GET /api/v1/users/{id}	Get profile by profile id	X-Auth-User-Id	404 UserProfileNotFoundException
GET /api/v1/users/auth/{authUserId}	Get profile by auth user id	X-Auth-User-Id	Used by frontend after login
PUT /api/v1/users/{id}	Update profile	X-Auth-User-Id	Ownership enforced
DELETE /api/v1/users/{id}	Delete profile	X-Auth-User-Id	Ownership enforced

Internal (Feign-only) endpoints — not routed by the gateway:

GET /api/v1/internal/users/{authUserId}	Profile lookup for other services	Service-to-service	Consumed by complaint-service & auth-service
---	-----------------------------------	--------------------	--

4.4 Complaint Service — complaint-service (port 8083)

POST /api/v1/complaints — create complaint with images (multipart)

Auth	X-Auth-User-Id (citizen)
Body	multipart/form-data — parts: title, description, latitude, longitude (all required), images (optional, up to several files)
File rules	Content type JPG/PNG/WEBP; ≤ 10 MB per file (multipart max-file-size 10 MB, max-request-size 40 MB)
Success	201 — ComplaintResponse (status PENDING, imageUrls, history with first entry)
Errors	400 invalid file type/size · 404 user not found (Feign to user-service) · 500 Cloudinary failure

GET /api/v1/complaints — list complaints

Auth	X-Auth-User-Id
Query	userId (optional) — when present, returns findByCreatedBy(userId) (own complaints)
Success	200 — List<ComplaintResponse>

GET /api/v1/complaints/{id} — complaint detail

Auth	X-Auth-User-Id
Success	200 — ComplaintResponse incl. history entries and image URLs
Errors	404 ComplaintNotFoundException

PATCH /api/v1/complaints/{id}/status — update status (authority path)

Auth	X-Auth-User-Id
Body	{ "status": "IN_PROGRESS", "priority": "HIGH", "remarks": "..." } — status required; priority/remarks optional
Success	200 — ComplaintResponse
Errors	400 ResolutionProofRequiredException if status = RESOLVED · 404 not found

DELETE /api/v1/complaints/{id} — delete complaint

Auth	X-Auth-User-Id
Success	200 — void (also publishes delete event)

Admin endpoints — /api/v1/complaints/admin

Method / Path	Purpose	Auth / Role	Notes
PUT /{id}/assign	Assign complaint to a field officer	ADMIN (role re-verified via Feign)	Body {"fieldOfficerId": 2}; target must be FIELD_OFFICER; terminal complaints immutable
POST /{id}/resolve	Resolve complaint with proof photo	ADMIN (role re-verified via Feign)	Multipart: proofImage (required, JPG/PNG/WEBP ≤ 10 MB) + optional remarks/priority

Field-officer endpoints — /api/v1/complaints/field-officer

Method / Path	Purpose	Auth / Role	Notes
GET /	List complaints assigned to me	FIELD_OFFICER (role re-verified via Feign)	findByAssignedTo(officerUserId)
PATCH /{id}/status	Update status of my complaint	FIELD_OFFICER + ownership	Same body as authority path; RESOLVED rejected here too
POST /{id}/resolve	Resolve with proof photo	FIELD_OFFICER + ownership	Multipart as admin resolve; ownership enforced

Example — successful complaint creation response:

```
{
  "success": true,
  "message": "Complaint created successfully",
  "data": {
    "id": 42,
    "title": "Sewage overflow near Indira Park",
    "description": "Manhole overflow for 3 days",
    "latitude": 17.4325,
    "longitude": 78.4567,
    "status": "PENDING",
    "priority": "MEDIUM",
    "createdBy": 7,
    "assignedTo": null,
    "resolutionRemarks": null,
    "resolutionProofImageUrl": null,
    "imageUrls": ["https://res.cloudinary.com/<cloud>/image/upload/v1/complaints/abc123.jpg"],
    "history": [{ "status": "PENDING", "remarks": null, "updatedBy": 7, "updatedAt": "2026-08-18T09:15:00" }],
    "createdAt": "2026-08-18T09:15:00",
    "updatedAt": "2026-08-18T09:15:00"
  }
}
```

4.5 Notification Service — notification-service (port 8085)

Method / Path	Purpose	Auth	Notes
GET /api/v1/notifications	Paged notification feed	X-Auth-User-Id	Query: page (0), size (clamped 1–100); newest first; scoped to caller
GET /api/v1/notifications/unread-count	Unread badge count	X-Auth-User-Id	Returns Long
PATCH /api/v1/notifications/{id}/read	Mark one notification read	X-Auth-User-Id	404 if not found or not owned
PATCH /api/v1/notifications/read-all	Mark all read	X-Auth-User-Id	Returns number marked
DELETE /api/v1/notifications/{id}	Delete notification	X-Auth-User-Id	Soft delete (deleted=true)

NotificationResponse fields: id, userId, title, message, notificationType, referenceId, referenceType, read, readAt, metadata, createdAt, updatedAt.

4.6 Community Service — community-service (port 8084)

Five resource groups, all with the same CRUD shape (POST create with @Valid body, GET list, GET /{id}, PUT /{id}, DELETE /{id}) plus group-specific extras:

Group	Base path	Auth (write)	Extras
Events	/api/v1/events	ADMIN	GET /upcoming; POST /{id}/register (citizen, duplicate + capacity checks)
Articles	/api/v1/articles	ADMIN	GET /latest — merges DB articles with live GNews feed (keyword "water", country IN, en)
NGOs	/api/v1/ngos	ADMIN	GET /search?keyword=&lat=&lng= — Google Places Nearby Search + Place Details; Overpass fallback when no API key
Pipelines	/api/v1/pipelines	ADMIN	—
Treatment plants	/api/v1/treatment-plants	ADMIN	GET /external — Telangana Govt ArcGIS open data (tgrac.telangana.gov.in)
Lakes	/api/v1/lakes	ADMIN	GET /external — OpenStreetMap Overpass API (Hyderabad bbox)

Reads (GET) are public; writes require ADMIN (role re-verified via Feign to auth-service, mirroring complaint-service).

4.7 Internal API surface (Feign, not gateway-routed)

Endpoint	Owner	Consumed by	Purpose
----------	-------	-------------	---------

GET /api/v1/internal/auth/users/{userId}/role	auth-service (AuthInternalController)	complaint-service	Role re-verification (ADMIN / FIELD_OFFICER)
GET /api/v1/internal/users/{authUserId}	user-service (InternalUserController)	complaint-service, auth-service	User existence + profile lookup
POST /api/v1/users/internal/profile	user-service (UserController)	auth-service	Best-effort profile creation at registration

Authorization model (verified): only auth-service installs a Spring Security filter chain (JWT validation). The other services rely on the gateway to route traffic and on the trusted `X-Auth-User-Id` header, then re-verify roles via synchronous Feign calls to the internal endpoints above when an operation is role-sensitive. There is no gateway-level token validation filter — the JWT is validated inside auth-service, and complaint/community re-check roles over Feign.

5. Swagger / OpenAPI

5.1 Configuration

Each backend service includes `springdoc-openapi-starter-webmvc-ui` (auth, user, complaint, community, notification) and the gateway uses `springdoc-openapi-starter-webflux-ui` with `springdoc.swagger-ui.urls` aggregation. Configuration is centralized in per-service `OpenApiConfig` classes (e.g. `com.sewagealert.auth.config.OpenApiConfig`), which build an `OpenAPI` bean with:

- **Title / description / version** — e.g. "Sewage Alert Auth Service API".
- **Global Bearer security scheme** — a JWT bearer scheme registered so every secured endpoint shows the padlock; endpoints tagged public are documented as unauthenticated.
- **Grouped paths** — controllers are tagged by feature (auth, complaints, notifications, events, articles, ngos, pipelines, treatment-plants, lakes).

The commit "*feat(auth): allow unauthenticated Swagger documentation*" made the UI and OpenAPI JSON publicly reachable (the swagger paths are added to the permit-all list in the security filter chain) so documentation remains browsable without logging in.

5.2 Accessing the docs

Service	Swagger UI (direct)	OpenAPI JSON
api-gateway (aggregated)	http://localhost:8080/swagger-ui.html	<code>/v3/api-docs</code> (aggregated)
auth-service	http://localhost:8081/swagger-ui.html	http://localhost:8081/v3/api-docs
user-service	http://localhost:8082/swagger-ui.html	<code>/v3/api-docs</code>
complaint-service	http://localhost:8083/swagger-ui.html	<code>/v3/api-docs</code>
community-service	http://localhost:8084/swagger-ui.html	<code>/v3/api-docs</code>
notification-service	http://localhost:8085/swagger-ui.html	<code>/v3/api-docs</code>

The gateway's `springdoc.swagger-ui.urls` property aggregates every service's `/v3/api-docs` into one UI, so <http://localhost:8080/swagger-ui.html> is the single entry point to explore all APIs. In production the exact host is the deployed gateway address (see §15).

5.3 Testing with JWT inside Swagger

1. Open the UI and call `POST /api/v1/auth/login` with a verified account (e.g. the seeded admin `admin@sewagealert.com` in dev).
2. Copy the `token` value from the response.
3. Click **Authorize** (top right) and paste `Bearer <token>`.
4. Secured auth endpoints are now callable; for other services add the `X-Auth-User-Id` header manually if needed (documented per endpoint).

Important request/response schemas exposed by springdoc include `RegisterRequest`, `LoginRequest`, `AuthResponse`, `ComplaintRequest`, `ComplaintResponse`, `NotificationResponse`, `ApiResponse` and the domain enums (`Role`, `ComplaintStatus`, `ComplaintPriority`, `NotificationType`).

6. Spring Boot Code Architecture

All seven services are Spring Boot 3.5 applications on Java 25, organized in a Maven multi-module reactor (`pom.xml` at the root declares each service as a module; services share a common `<parent>` with dependency management). Every service follows the same layered package layout under `com.sewagealert.<service>`:

```
controller/ → REST entry points (thin, delegate to services)
dto/        → request/response records & classes (validation annotations on requests)
service/    → interfaces
service/impl/ → business logic implementations
model/      → JPA entities
repository/ → Spring Data JPA repositories (interface-only)
client/     → OpenFeign clients to other services
producer/   → RabbitMQ publishers (interface + impl)
config/     → @Configuration / @ConfigurationProperties classes
security/   → auth-service only: SecurityConfig, JwtAuthenticationFilter, JwtTokenProvider
exception/  → GlobalExceptionHandler + custom exceptions
util/       → helpers (e.g. VerificationTokenGenerator)
consumer/   → notification-service only: RabbitMQ listeners
```

6.1 The request pipeline (controller → DB → response)

Using complaint creation as the worked example — `ComplaintController.createComplaint()`:

Layer	Class (actual)	Responsibility
Controller	<code>ComplaintController</code>	Maps HTTP → service; extracts X-Auth-User-Id header; binds multipart <code>ComplaintRequest</code> parts; returns <code>ResponseEntity<ApiResponse<ComplaintResponse>></code>
DTO (request)	<code>ComplaintRequest</code>	Carries validated fields (title, description, latitude, longitude); Bean Validation via <code>@NotBlank</code> / <code>@NotNull</code>
Service interface	<code>ComplaintService</code>	Contract: create/getAll/getById/updateStatus/assign/resolve/delete
ServiceImpl	<code>ComplaintServiceImpl</code>	Orchestrates: Feign user check → image validation → Cloudinary upload → <code>@Transactional</code> save with rollback-sync image cleanup → history entry → publish event
Mapper	<code>ComplaintResponse.fromEntity(...)</code>	Static factory mapping entity → response DTO (flattens images to URL list, builds <code>HistoryEntry</code> DTOs)
Repository	<code>ComplaintRepository</code> (extends <code>JpaRepository</code>)	<code>findById</code> , <code>findAll</code> , <code>findByCreatedBy</code> , <code>findByAssignedTo</code> , <code>findByIdWithImages</code> (fetch-join) — Spring Data derives queries from method names
Entity	<code>Complaint</code> , <code>ComplaintImage</code> , <code>ComplaintHistory</code>	JPA <code>@Entity</code> mapped to <code>complaints</code> , <code>complaint_images</code> , <code>complaint_history</code> ; one-to-many relations with cascade
Producer	<code>NotificationEventProducerImpl</code>	After DB commit, publishes a <code>NotificationEvent</code> to <code>notification.exchange</code> (fire-and-forget)

6.2 Dependency injection & how Spring wires it

- Constructor injection everywhere** — controllers/services take interfaces in their constructors (e.g. `ComplaintController(ComplaintService complaintService)`); Spring resolves the single `@Service` implementation.
- Interfaces + impls** — `ComplaintService` ← `@Service ComplaintServiceImpl`; `NotificationEventProducer` ← `NotificationEventProducerImpl`; this mirrors the same pattern in auth/community/notification services.
- Configuration properties** — `@ConfigurationProperties` beans (`RabbitMqProperties`, `UploadProperties`, `CloudinaryConfig`, `EmailJsProperties`, `AppProperties`) bind `app.*/cloudinary.*/emailjs.*` prefixes instead of scattered `@Value`.
- Feign clients** — `@FeignClient(name = "user-service")` / `@FeignClient(name = "auth-service")` interfaces are injected like local beans and resolve targets through Eureka (service name → instance).
- Transactions** — `@Transactional` on service methods; complaint image cleanup on rollback uses Spring's `TransactionSynchronization` (registered synchronizations run after rollback to delete already-uploaded Cloudinary files).

6.3 Cross-cutting pieces in every service

Piece	Pattern used	Example classes
Uniform response	<code>ApiResponse<T></code> envelope: success / message / data / errors	<code>com.sewagealert.*.common.ApiResponse</code> (per service)
Exception handling	<code>@RestControllerAdvice</code> maps exceptions → HTTP status; <code>@ExceptionHandler</code> methods per exception	<code>GlobalExceptionHandler</code> in every service

Custom exceptions	Runtime exceptions with clear names	<code>ComplaintNotFoundException</code> , <code>ResolutionProofRequiredException</code> , <code>EmailAlreadyExistsException</code> , <code>InvalidCredentialsException</code>
Event publishing	Producer interface + impl using <code>RabbitTemplate.convertAndSend</code>	<code>NotificationEventProducerImpl</code> (auth, complaint)
Service-to-service	OpenFeign + Eureka discovery	<code>UserServiceClient</code> , <code>AuthServiceClient</code>
External HTTP	OpenFeign clients with per-client timeouts for public APIs	<code>NewsApiClient</code> , <code>GooglePlacesClient</code> , <code>OverpassClient</code> , <code>TelanganaStpClient</code> , <code>EmailJsClient</code>

7. Spring Security & JWT

Verified scope: Spring Security is configured *only* in auth-service. The gateway does not validate JWTs, and the other services authenticate callers via the trusted `X-Auth-User-Id` header plus synchronous role re-verification through Feign. This section therefore documents the real, working JWT chain in auth-service.

7.1 Components (actual classes in `com.sewagealert.auth.security`)

Class	Role
<code>SecurityConfig</code>	<code>@Configuration + @EnableMethodSecurity</code> ; defines <code>SecurityFilterChain</code> , <code>PasswordEncoder</code> (BCrypt), <code>AuthenticationManager</code> ; stateless sessions; CSRF disabled; CORS allowed; permit-all for <code>/api/v1/auth/register login verify-code resend-verification</code> , H2 console and Swagger paths
<code>JwtAuthenticationFilter</code>	<code>OncePerRequestFilter</code> ; extracts <code>Authorization: Bearer ...</code> , validates the token via <code>JwtTokenProvider</code> , and on success sets a <code>UsernamePasswordAuthenticationToken</code> (principal = <code>userId</code> , authority = <code>ROLE_<role></code>) into the <code>SecurityContextHolder</code> ; on failure logs and continues (unauthenticated)
<code>JwtTokenProvider</code>	Generates and validates JWTs with <code>jwt</code> ; reads <code>app.jwt.secret</code> and <code>app.jwt.expiration-ms</code> (24 h); claims: <code>sub = userId</code> , <code>email</code> , <code>role</code> ; signs HS256; parses with <code>verifyWith</code> key
<code>AuthServiceImpl</code>	Implements the actual login logic: <code>findByEmail</code> → <code>passwordEncoder.matches</code> → <code>emailVerified</code> check → <code>jwtTokenProvider.generateToken(userId, email, role)</code> . No <code>UserDetailsService/AuthenticationManager.authenticate</code> usage — authentication is a direct DAO check (verified in code).

7.2 Password hashing

```
@Bean
public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); } // SecurityConfig
// usage in AuthServiceImpl.register():
user.setPassword(passwordEncoder.encode(request.getPassword()));
```

7.3 JWT structure (what is actually signed)

```
Header    { "alg": "HS256", "typ": "JWT" }
Payload   { "sub": "<userId>", "email": "<email>", "role": "CITIZEN", "iat": ..., "exp": +24h }
Signature HMACSHA256(base64url(header) + "." + base64url(payload), app.jwt.secret)
```

Config (values in source are defaults/dev-only; replace in production): `app.jwt.secret` — a base64-encoded dev secret; `app.jwt.expiration-ms`: 86400000.

7.4 Authentication / authorization flow

```
Frontend
├─ POST /api/v1/auth/login {email, password}
└─
  AuthServiceImpl.login()
  ├─ findByEmail(email) —X— InvalidCredentialsException → 401
  ├─ passwordEncoder.matches(password, hash) —X— 401
  ├─ user.isEmailVerified() —X— 401 (EMAIL_NOT_VERIFIED)
  └─ generateToken(userId, email, role) → 200 AuthResponse{token, ...}

Frontend stores token (localStorage key "sa_token") and sends on every call:
  Authorization: Bearer <jwt> + X-Auth-User-Id: <id>

JwtAuthenticationFilter (auth-service, on protected routes)
├─ extract Bearer token → JwtTokenProvider.validateToken()
├─ │   └─ parseSignedClaims / verifyWith(secret key)
├─ │   └─ expired or tampered → log, skip (request continues unauthenticated)
└─ valid → SecurityContextHolder.getContext().setAuthentication(
      UsernamePasswordAuthenticationToken(userId, null, [ROLE_<role>]))

Controller layer
├─ Authentication principal = userId (e.g. GET /api/v1/auth/profile)
├─ @PreAuthorize("hasRole('ADMIN')") or requireAdmin() on admin endpoints
└─ other services: X-Auth-User-Id header + Feign role check (complaint-service
      AdminComplaintController.verifyRole(id, "ADMIN") → AUTH-SERVICE internal endpoint)
```

7.5 Role-based access control (verified)

Role	Granted at	Capabilities in code
------	------------	----------------------

CITIZEN	Registration (default)	Create complaint, view own complaints, register for events, notification feed
ADMIN	Seeded (<code>AdminSeeder</code> on startup: <code>admin@sewagealert.com</code>)	Assign complaints, resolve with proof, field-officer list, community content CRUD
FIELD_OFFICER	Seeded (<code>FieldOfficerSeeder</code> : 3 demo officers)	View assigned complaints, update status, resolve with proof (ownership enforced)

7.6 Public vs protected endpoints

Category	Endpoints
Public (permit-all)	<code>/api/v1/auth/register</code> , <code>/login</code> , <code>/verify-code</code> , <code>/resend-verification</code> ; H2 console; Swagger UI + <code>/v3/api-docs</code> ; all community GET endpoints
Protected (JWT, auth-service)	<code>/api/v1/auth/profile</code> , <code>/api/v1/auth/admin/field-officers</code>
Protected (X-Auth-User-Id + role re-check)	Complaint create/update/delete, admin + field-officer endpoints, community write endpoints, notifications, user profile writes

7.7 Invalid / expired token behaviour

- **auth-service:** the filter logs the validation failure and leaves the request unauthenticated; protected endpoints then reject it (401). The frontend listens for 401 responses and clears the session (event `sa:auth-expired` → redirect to login).
- **Other services:** they trust `X-Auth-User-Id`; role verification failures surface as 403/404 from the Feign-backed checks (and `GlobalExceptionHandler` converts Feign 401/403 into the corresponding error response).

8. Database Architecture

8.1 Databases used (verified)

Service	Local/dev default	Docker/prod (MySQL 8.4)	Schema
auth-service	H2 in-memory (sewagealert_auth)	MySQL	sewagealert_auth
user-service	H2 in-memory (sewagealert_users)	MySQL	sewagealert_users
complaint-service	MySQL local (Workbench)	MySQL	sewagealert_complaints
community-service	H2 in-memory (sewagealert_community)	MySQL	sewagealert_community
notification-service	MySQL local (Workbench)	MySQL	sewagealert_notifications

Database-per-service: each microservice owns its schema (no cross-service foreign keys). Cross-service lookups happen over HTTP/Feign, and `docker/mysql/init/01-create-databases.sql` creates all five databases on first container boot. Tables are created/updated automatically via `Hibernate ddl-auto: update`.

8.2 Connection flow & JPA configuration

- **Driver / dialect:** MySQL services use `com.mysql.cj.jdbc.Driver` + `org.hibernate.dialect.MySQLDialect`; H2 services use H2 defaults.
- **URL params:** `useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true`; containerized services are pointed at `jdbc:mysql://mysql:3306/...` via `SPRING_DATASOURCE_URL` env overrides.
- **JPA:** `ddl-auto: update`, `show-sql: true` (dev), `open-in-view: false` (auth), formatted SQL; repositories extend `JpaRepository<Entity, Long>`; queries are derived from method names or `@Query` fetch-joins.
- **Transactions:** `@Transactional` on service methods; complaint image workflow registers rollback synchronizations to delete Cloudinary objects on DB failure (compensation).

8.3 Entities and relationships (ER overview)

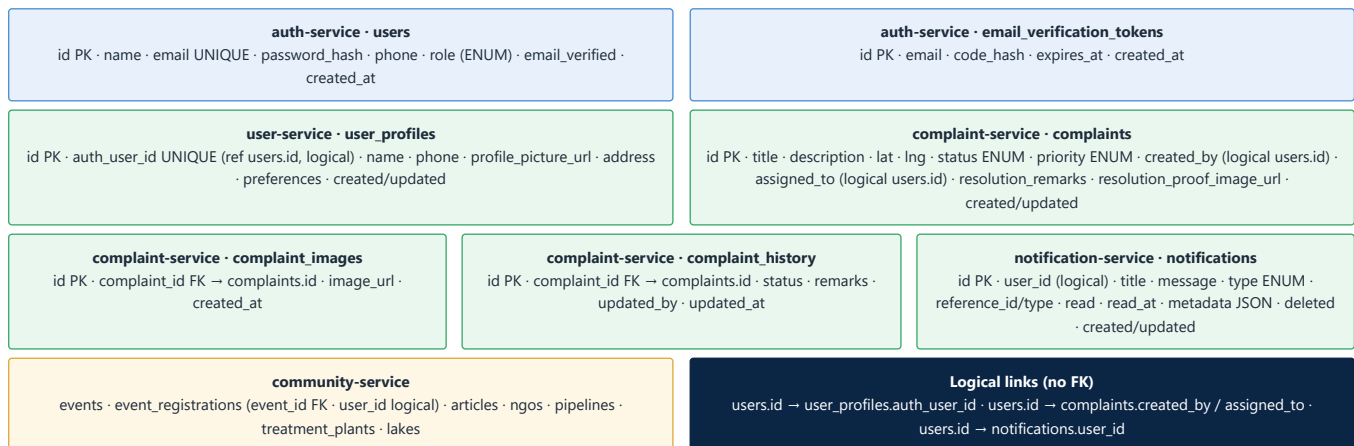


Figure 8.1 — Entity-relationship overview across the five databases (FKs shown within a service; cross-service links are logical references resolved via Feign).

8.4 Key repository operations

Repository	Derived/annotated queries (actual)
ComplaintRepository	<code>findAll</code> , <code>findById</code> , <code>findByCreatedBy</code> , <code>findByAssignedTo</code> , <code>findByIdWithImages</code> (@Query fetch-join images)
NotificationRepository	Paged find for a user (newest first), <code>countByUserIdAndReadFalse</code> (unread badge), soft-delete updates, mark-all-read bulk update
UserRepository (auth)	<code>existsByEmail</code> , <code>findByEmail</code> , <code>findById</code>
UserProfileRepository	<code>findByAuthUserId</code> , <code>existsByAuthUserId</code>
Community repositories	CRUD + <code>EventRegistrationRepository.existsByEventIdAndUserId</code> (duplicate registration guard), upcoming-events query

8.5 Data lifecycle

- **Write path:** Controller → ServiceImpl (@Transactional) → repository.save → Hibernate INSERT/UPDATE; complaint history is appended on every status change (audit trail).
- **Read path:** repository derived queries → entities → `fromEntity` DTO mapping → JSON envelope.
- **Soft delete:** notifications keep rows (`deleted=true`) so unread counts and feeds remain consistent; profiles/complaints use hard delete.
- **Seed data:** auth-service seeds one admin + three field officers on startup (`AdminSeeder`, `FieldOfficerSeeder`); community-service seeds demo events (`EventSeeder`).

9. RabbitMQ Architecture

9.1 Why RabbitMQ

RabbitMQ decouples business events from their side effects (notifications, emails). Complaint and auth services publish domain events *fire-and-forget* — a broker outage never blocks complaint creation or registration (failures are logged by the producer). The notification service consumes, persists, and forwards events; future channels (SMS/push) can be added without touching producers.

9.2 Topology (declared by Notification Service — `RabbitMqConfig`)

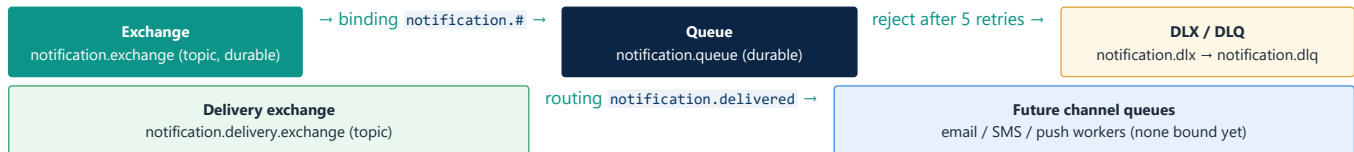


Figure 9.1 — RabbitMQ topology. Main queue's DLX header `x-dead-letter-exchange → notification.dlx` routes permanently failed messages to the DLQ.

Object	Name	Declared by
Exchange (topic, durable)	notification.exchange	notification-service <code>RabbitMqConfig</code>
Queue (durable)	notification.queue	notification-service (bound with <code>notification.#</code>)
Dead-letter exchange	notification.dlx	notification-service
Dead-letter queue	notification.dlq (routing key <code>notification.dlx</code>)	notification-service
Delivery exchange	notification.delivery.exchange (routing key <code>notification.delivered</code>)	notification-service (no queue bound yet — ready for email/SMS/push workers)

9.3 Routing keys actually published

Routing key	Producer (service)	Event
notification.user.registered	auth-service	New account created → verification email + in-app notification
notification.user.verified	auth-service	Email verified → welcome email (optional) + in-app notification
notification.verification.requested	auth-service	OTP resent → verification email
notification.created	complaint-service	Complaint created → notify citizen
notification.assigned	complaint-service	Complaint assigned → notify field officer
notification.updated / status variants	complaint-service	Status changed (IN_PROGRESS, REJECTED, REOPENED, ...)
notification.resolved	complaint-service	Complaint resolved → notify citizen
notification.deleted	complaint-service	Complaint deleted

The binding pattern `notification.#` (hash) intentionally matches both two-word keys (`notification.created`) and three-word keys (`notification.user.registered`) — a single `*` star would silently drop the three-word events (documented in `RabbitMqProperties.routingKeyPattern`).

9.4 Message shape — `NotificationEvent` DTO

```
{
  "eventId": "uuid",           // generated per publish
  "eventType": "USER_REGISTERED", // enum (EventTypeResolver maps key → type)
  "userId": 7,                // recipient
  "title": "...", "message": "...",
  "metadata": { "email": "...", "name": "...", "verificationCode": "123456", ... }
}
```

Serialization is Spring AMQP's default `Jackson2JsonMessageConverter` (JSON). The consumer `NotificationEventConsumer.processEvent()` validates the payload, resolves the `NotificationType`, **strips verificationCode from metadata before persisting**, saves the notification, triggers emails for auth events, then re-publishes the stored notification to the delivery exchange.

9.5 Reliability & error handling

- **Container retry:** listener retry enabled — exponential backoff, initial 1 s, multiplier 2.0, max 5 attempts, max 10 s.
- **Requeue policy:** `acknowledge-mode: auto`, `default-requeue-rejected: false` — after max attempts the message is rejected without requeue and routed to `notification.dlq`.
- **DLQ consumer:** `DeadLetterQueueConsumer` listens on `notification.dlq`, logs the failed event with its eventId for inspection/replay.
- **Producer resilience:** publishing is wrapped so exceptions are logged, never propagated — the API call already succeeded.

9.6 Synchronous REST vs asynchronous RabbitMQ (when each is used)

	Synchronous REST (Feign/Eureka)	Asynchronous RabbitMQ
Used for	Queries and decisions that must complete before responding: user existence checks, role re-verification, profile creation at registration	Side effects after a committed business action: notifications, verification/welcome emails
Latency	Added to request time (timeouts: Feign defaults, external clients 5–45 s)	Near-zero to request; consumer processes independently
Failure impact	Request fails → 4xx/5xx to caller	Logged; retried 5×; DLQ for manual replay; business action unaffected
Example	<code>ComplaintServiceImpl</code> → Feign <code>AuthServiceClient.getUserRole()</code> before assigning a complaint	After complaint save → <code>convertAndSend("notification.exchange", "notification.created", event)</code>

10. Notification & Email Flow

End-to-end notification pipeline (all stages verified in code):

```
Business event (complaint created/assigned/resolved, user registered/verified)
→ Producer (auth/complaint service, NotificationEventProducerImpl)
→ RabbitMQ notification.exchange → notification.queue
→ NotificationEventConsumer.processEvent()
→ NotificationServiceImpl (validate → resolve type → strip secrets → persist)
→ in-app notification (notification-service DB) → user's feed (GET /api/v1/notifications)
→ EmailNotificationServiceImpl → EmailJsClient → EmailJS API → user's inbox
```

10.1 Notification event structure & type resolution

Piece	Detail
Event DTO	<code>NotificationEvent</code> : <code>eventId</code> (UUID), <code>eventType</code> , <code>userId</code> , <code>title</code> , <code>message</code> , <code>metadata</code> (Map)
Routing-key → type mapping	<code>EventTypeResolver</code> maps the RabbitMQ routing key to a <code>NotificationType</code> enum (USER_REGISTERED, EMAIL_VERIFIED, COMPLAINT_CREATED, COMPLAINT_ASSIGNED, COMPLAINT_UPDATED, COMPLAINT_RESOLVED, COMPLAINT_REJECTED, COMPLAINT_REOPENED, ...)
Stored model	<code>Notification</code> entity: <code>userId</code> , <code>title</code> , <code>message</code> , <code>NotificationType</code> , <code>referenceId/referenceType</code> (e.g. complaint id), <code>read/readAt</code> , <code>metadata</code> (JSON map), <code>deleted</code> flag
API projection	<code>NotificationResponse</code> (see §4.5)

10.2 Recipient determination

- Each published event carries the **target user id** in `NotificationEvent.userId`: citizen for created/resolved/updated/deleted events, the assigned **field officer** for `notification.assigned`.
- Email recipients are derived from the event's `metadata.email` (auth events always include it). Complaint events are in-app only — no email is sent for them.

10.3 Email sending — EmailJS (actual integration)

Item	Detail
Client	<code>EmailJsClient</code> — Feign client → POST <code>https://api.emailjs.com/api/v1.0/email/send</code>
Config	<code>EmailJsProperties</code> binds <code>emailjs.service-id/template-id/public-key/private-key/welcome-template-id</code> (env: <code>EMAILJS_*</code>)
Verification email	Triggered by <code>USER_REGISTERED</code> / <code>EMAIL_VERIFICATION_REQUESTED</code> events; template params: <code>name</code> , <code>email</code> , <code>verification_code</code> (OTP from metadata)
Welcome email	Triggered by <code>EMAIL_VERIFIED</code> ; params: <code>name</code> , <code>email</code> , <code>login_url</code> (= <code>FRONTEND_URL</code> + <code>/login</code>); only sent when <code>EMAILJS_WELCOME_TEMPLATE_ID</code> is set
HTTP timeouts	Connect 5 s, read 15 s (<code>SimpleClientHttpRequestFactory</code>) — a hung provider never stalls the consumer thread

10.4 Success / failure handling

- Storage failure:** the consumer's DB failure triggers RabbitMQ retry (5 attempts, backoff) then DLQ — the notification is never silently lost.
- Email failure:** logged with `eventId`, never rethrown — the persisted notification and message ack are preserved; users can always request a resend (POST `/api/v1/auth/resend-verification`).
- Missing credentials:** without `EMAILJS_*` env vars the service still runs; emails are skipped with a warning log (verified in `EmailNotificationServiceImpl`).
- Delivery fan-out (future-ready):** after persisting, each notification is re-published to `notification.delivery.exchange` with key `notification.delivered` — no queue is bound yet, so dedicated email/SMS/push workers can attach later without touching producers or the main flow.

11. File / Image Upload Architecture

11.1 Storage mechanism — Cloudinary (actual integration)

Complaint images are uploaded to **Cloudinary** (object/image CDN); only the returned `secure_url` strings are stored in MySQL. There is no local file storage or S3 — verified in `CloudinaryStorageService` and `CloudinaryConfig`.

Item	Detail
Client config	<code>CloudinaryConfig</code> binds <code>cloudinary.cloud-name/api-key/api-secret</code> from env (<code>CLOUDINARY_CLOUD_NAME</code> , <code>CLOUDINARY_API_KEY</code> , <code>CLOUDINARY_API_SECRET</code>)
Upload API	<code>CloudinaryStorageService.uploadFile(MultipartFile)</code> → <code>cloudinary.uploader().upload(bytes, ObjectUtils.asMap("folder", "complaints"))</code> → returns <code>secure_url</code>
Validation	Content type must be JPG / PNG / WEBP; ≤ 10 MB per file (<code>UploadProperties.max-file-size</code> , mirrors <code>spring.servlet.multipart.max-file-size: 10MB</code> ; request ceiling 40 MB)
Multipart limits	Gateway raises its codec buffer to 40 MB so uploads pass through; each service sets its own multipart limits
DB reference	<code>ComplaintImage.imageUrl</code> rows (one per image) + <code>Complaint.resolutionProofImageUrl</code> column

11.2 Resolution-proof workflow (v1.1.0 mandatory)

1. Admin or field officer calls `POST /api/v1/complaints/{admin|field-officer}/{id}/resolve` with `multipart/form-data: proofImage` + optional `remarks/priority`.
2. `ComplaintServiceImpl.applyResolution()` throws `ResolutionProofRequiredException` (400) if the proof image is missing/empty — a complaint can never be marked RESOLVED without one (the JSON status PATCH rejects RESOLVED outright).
3. The proof is uploaded *first*; on any later DB failure a rollback synchronization deletes the just-uploaded proof from Cloudinary (compensation, same pattern as creation).
4. Status → `RESOLVED`, `resolutionProofImageUrl` set, remarks stored, history entry appended, `notification.resolved` published.

11.3 Retrieval / display & security considerations

- Images are **not** proxied by the backend — the frontend loads `imageUrls` / `resolutionProofImageUrl` directly from Cloudinary's CDN URLs returned in responses.
- Uploads go only to the fixed `complaints` folder; file bytes are validated by content type before upload (no arbitrary uploads); Cloudinary credentials are env-only and never committed.
- Ownership: images are only exposed through the owning complaint's detail/list responses.
- Without Cloudinary credentials the services start fine but uploads fail at runtime with a logged error — no partial state is left behind thanks to the rollback sync.

12. API Gateway & Service-to-Service Communication

12.1 Gateway responsibility — Spring Cloud Gateway (WebFlux)

`api-gateway` (port **8080**) is the **single entry point** for all frontend traffic. It does not run business logic; it routes by path prefix and aggregates Swagger. Configuration lives in `application.yml` (`spring.cloud.gateway`):

Route id	Path predicate	Target (lb://)
auth-service	<code>/api/v1/auth/**</code>	<code>lb://AUTH-SERVICE</code>
user-service	<code>/api/v1/users/**</code>	<code>lb://USER-SERVICE</code>
complaint-service	<code>/api/v1/complaints/**</code>	<code>lb://COMPLAINT-SERVICE</code>
community-service-events ... lakes	<code>/api/v1/events articles ngos pipelines treatment-plants lakes/**</code>	<code>lb://COMMUNITY-SERVICE</code>
notification-service	<code>/api/v1/notifications/**</code>	<code>lb://NOTIFICATION-SERVICE</code>

- **Service discovery:** routes use `lb://SERVICE-NAME` — the gateway resolves instances from the **Eureka** registry (load-balanced).
- **Codec:** `spring.codec.max-in-memory-size: 40MB` so multipart complaint uploads (up to 40 MB) pass through without `DataBufferLimitException`.
- **CORS:** global CORS allowing all origins/methods/headers, plus `DedupeResponseHeader=Access-Control-Allow-Origin`, `RETAIN_UNIQUE` default filter to avoid duplicate CORS headers.
- **Auth handling:** the gateway is stateless and transparent — it forwards `Authorization` and `X-Auth-User-Id` headers unchanged; JWT validation happens in `auth-service`, role checks in the target services.
- **Swagger aggregation:** `springdoc webflux` config exposes a single UI listing all services' OpenAPI docs.

12.2 Service-to-service communication (internal)

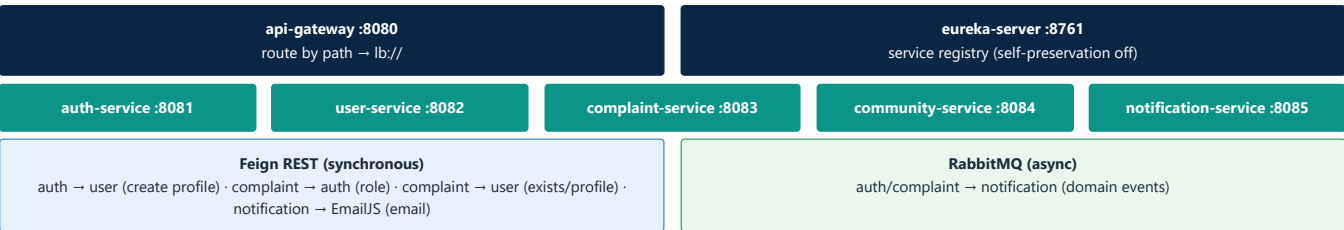


Figure 12.1 — Runtime topology: gateway + Eureka + five business services; Feign calls for decisions, RabbitMQ for side effects.

Caller	Feign client	Target	Purpose
auth-service	<code>UserServiceClient</code>	user-service	Best-effort profile creation on registration; profile lookups
complaint-service	<code>UserServiceClient</code>	user-service	Verify complainant exists before creating a complaint
complaint-service	<code>AuthServiceClient</code>	auth-service	<code>GET /api/v1/internal/auth/users/{id}/role</code> — re-verify ADMIN / FIELD_OFFICER on sensitive operations
notification-service	<code>EmailJSClient</code>	api.emailjs.com (external)	Send transactional emails
community-service	<code>NewsApiClient</code> , <code>GooglePlacesClient</code> , <code>OverpassClient</code> , <code>TelanganaStpClient</code>	external public APIs	Live articles, NGO search, lake geo-data, STP locations

Why both mechanisms: synchronous Feign is used where the caller needs an immediate decision or data (role checks, existence checks) — the response depends on it. RabbitMQ is used for fire-and-forget side effects so a downstream failure or broker outage never breaks the primary business transaction.

13. Docker Architecture

13.1 Dockerfiles

Each service has its own two-stage Dockerfile (`maven:3.9-eclipse-temurin-25` build → `eclipse-temurin:25-jre` runtime), building only the modules it needs (e.g. `auth-service` also copies `user-service/src` because of a compile-time dependency, and uses `mvn -pl auth-service -am clean package -DskipTests`). The root `Dockerfile` builds the whole reactor in one image and selects the service at runtime via `SERVICE_NAME`:

```
docker build -t sewage-alert-hyderabad .
docker run -e SERVICE_NAME=auth-service -p 8081:8081 sewage-alert-hyderabad
```

13.2 Compose topology

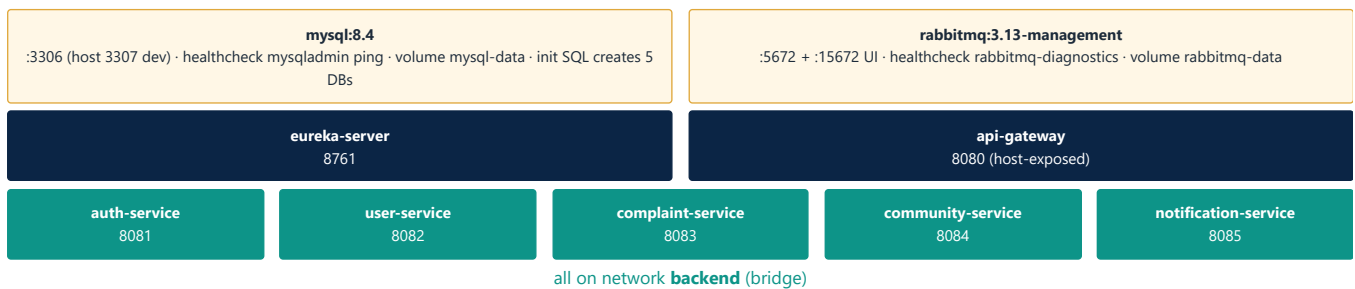


Figure 13.1 — docker-compose topology (identical shape in dev and prod compose files; prod pulls prebuilt images from Docker Hub, dev builds locally).

Aspect	Development (docker-compose.yml)	Production (docker-compose.prod.yml)
Images	Built locally (build: context: . → sewagealert/<svc>:latest)	Pulled from Docker Hub (devsiddhartha21/<svc>:latest) — no build on the server
Ports	Every service published to host (8081–8085) for debugging	Only GATEWAY_PORT (default 8080) exposed
MySQL	Host 3307 → container 3306; default password NewRoot@123	Not host-exposed; MYSQL_ROOT_PASSWORD required from .env
Secrets	Optional envs default to dev values (guest/guest, empty API keys)	Required envs with :? syntax (MYSQL_ROOT_PASSWORD, RABBITMQ_USERNAME/PASSWORD, FRONTEND_URL, EMAILJS_*) — compose fails fast if missing
Service config	DB/RabbitMQ overrides pointing at container names + env-style property overrides	Same env overrides; EUREKA_INSTANCE_PREFER_IP_ADDRESS: "true"
Restart	restart: unless-stopped	restart: unless-stopped

Environment variables passed to containers use Spring Boot's relaxed binding — e.g. `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, `SPRING_DATASOURCE_PASSWORD`, `RABBITMQ_HOST`, `EUREKA_CLIENT_SERVICEURL_DEFAULTZONE` — plus application-specific ones (`CLOUDINARY_*`, `GNEWS_API_KEY`, `GOOGLE_PLACES_API_KEY`, `EMAILJS_*`, `FRONTEND_URL`, `VERIFICATION_TOKEN_TTL_MINUTES`).

13.3 Startup order & health

- `depends_on` with `condition: service_healthy` for `mysql/rabbitmq` and `service_started` for `eureka` — services wait for their infrastructure before booting.
- Healthchecks: MySQL `mysqladmin ping` (10 s interval, 10 retries); RabbitMQ `rabbitmq-diagnostics ping` (10 s / 5 retries).
- Networks: single bridge network `backend`; volumes `mysql-data` and `rabbitmq-data` persist data.
- The gateway must start after Eureka (its routes resolve at runtime, so later restarts are fine).

14. CI/CD Pipeline

CI/CD is implemented with **GitHub Actions** in `.github/workflows/` — four workflows (verified): `backend-ci.yml`, `frontend-ci.yml`, `docker-build-push.yml`, and `qodana_code_quality.yml`.

14.1 Workflow inventory

Workflow	Trigger	Steps
Backend CI	push / PR → <code>main</code> , <code>develop</code>	checkout@v6 → setup-java@v4 (Temurin JDK 25, Maven cache) → <code>mvn --batch-mode clean verify</code> (build + run all tests)
Frontend CI	push / PR → <code>main</code> , <code>develop</code>	checkout@v6 → setup-node@v4 (Node 20, npm cache) → <code>npm ci</code> → <code>npm run lint --if-present</code> → <code>npm test --if-present</code> → <code>npm run build</code>
Docker Build & Push	push → <code>main</code> + manual dispatch	checkout → JDK 25 → <code>mvn clean verify (gate)</code> → login to Docker Hub (<code>DOCKERHUB_USERNAME</code> var + <code>DOCKERHUB_TOKEN</code> secret) → <code>buildx</code> → <code>build</code> & push all 7 service images, each tagged <code>latest + \${github.sha}</code>
Qodana	PR / push → <code>main</code> , <code>develop</code> (+ dispatch)	JetBrains Qodana scan with <code>QODANA_TOKEN</code> secret, annotations + PR comments

14.2 Pipeline flow diagram



Figure 14.1 — CI/CD flow: Developer → Git → CI (build/test) → Docker Build → Registry (Docker Hub) → EC2 deployment.

14.3 Key facts

- **Branching:** `main` (production) and `develop` (integration); feature branches push to `develop`; the `docker-build` workflow only runs on `main`.
- **Secrets handling:** Docker Hub credentials via GitHub Actions `secrets.DOCKERHUB_TOKEN` / `vars.DOCKERHUB_USERNAME`; Qodana via `secrets.QODANA_TOKEN`. No application secrets are stored in the repo (see §17).
- **Image tagging:** every service is published as both an immutable SHA tag (`...:${github.sha}`) and a mutable `latest`.
- **Deployment:** there is *no* automated deployment job in the workflows — deployment is the documented manual step of pulling the new images on the EC2 host with the production compose file (see §15). This is a known limitation (listed in Appendix A.6).
- **Testing:** backend runs the full Maven reactor tests; one notification-service integration test requires Docker (documented in §20); frontend tests run when present.

15. AWS / Cloud Architecture

Verified scope: the repository contains no Terraform, CloudFormation, ECS task definitions, IAM policies, or AWS SDK configuration. The only deployment references are: (1) the production compose file designed to run on a Docker host, and (2) Vercel configuration for the frontend (`frontend/vercel.json`). The EC2 target below is documented from the repository's compose/deployment design and README guidance — it is not backed by committed AWS infrastructure code.

15.1 What actually exists

Component	Where it appears	Role
EC2 (Docker host)	<code>docker-compose.prod.yml</code> usage model; README deployment section	Runs <code>docker compose</code> with prebuilt images; only gateway port 8080 exposed to the internet
Docker Hub (registry)	<code>docker-build-push.yml</code> pushes <code>devsiddhartha21/<service></code>	Holds the 7 service images (latest + SHA tags)
Vercel (frontend host)	<code>frontend/vercel.json</code>	Serves the React SPA with a rewrite proxy: <code>/api/* https://<EC2-host>/api/path*</code> (set via <code>VITE_API_BASE_URL</code> at build time); SPA fallback for deep routes
MySQL / RabbitMQ	Run as containers on the same host (compose)	Databases and message broker
Cloudinary / EmailJS / GNews / Google Places / Telangana ArcGIS / Overpass	External SaaS / public APIs	Image storage, transactional email, and community data sources

15.2 Deployment topology (as designed by the repo)

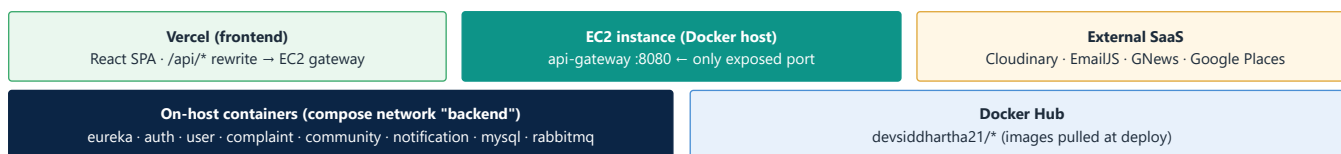


Figure 15.1 — Cloud deployment model: Vercel frontend + EC2 Docker host + Docker Hub registry + external SaaS.

15.3 Not used (verified absent)

- **No** AWS RDS, S3, ECS/Fargate, ECR, ELB, Route 53, CloudWatch, or IAM configuration exists in the repository.
- **No** AWS SDK dependencies or credentials in any `pom.xml` or config file.
- Image storage is Cloudinary; object storage is not AWS S3.

Deployment steps (from README / compose design): on the EC2 host, set the required env values in a deployment `.env`, then:

```
docker compose -f docker-compose.prod.yml --env-file .env pull
docker compose -f docker-compose.prod.yml --env-file .env up -d
```

Network security notes: only the gateway port is exposed; MySQL/RabbitMQ are internal to the bridge network; secrets are supplied via environment, never committed (no credentials appear anywhere in the repository).

16. Frontend Architecture

16.1 Framework & build

Item	Detail (verified)
Framework	React 18 + TypeScript, built with Vite; React Router v6 (BrowserRouter)
Key libraries	Leaflet + react-leaflet (hotspot maps, cluster markers, heatmap, Hyderabad boundary layer), Recharts (analytics), axios-free <code>fetch</code> -based client (<code>src/lib/api.ts</code>)
Dev server	Port 5173; proxies <code>/api</code> → <code>VITE_PROXY_TARGET</code> <code>http://localhost:8080</code> (gateway) — same-origin, no browser CORS
Prod routing	Vercel: <code>vercel.json</code> rewrites <code>/api/:path*/api/path*</code> and everything else → <code>/index.html</code> (SPA fallback for deep routes)
Bundling	Manual chunks: <code>vendor</code> (react/router), <code>charts</code> (recharts), <code>maps</code> (leaflet stack) for cache-friendly splits
Alias	@ → <code>src</code>

16.2 Component & page structure

```
src/
├─ App.tsx                # route table (public / citizen / officer / admin areas)
├─ main.tsx · index.css
├─ lib/
│  ├─ api.ts              # fetch client: session (sa_token/sa_user), ApiError, 401 → "sa:auth-expired"
│  ├─ auth.tsx            # AuthProvider + useAuth (login/register/logout/refreshProfile, role flags)
│  └─ toast.tsx · utils.ts · cn.ts · heatmap.ts · hyderabad.ts · hyderabad-boundary.ts · notifications.ts
├─ hooks/
├─ components/
│  ├─ layout/             # AppShell, AuthLayout, PublicLayout, RouteGuards (RequireAuth/Admin/FieldOfficer/RedirectIfAuthenticated)
│  ├─ admin/              # AssignOfficerModal, ResourceManager
│  ├─ complaints/         # ComplaintCard, ComplaintDetail
│  ├─ maps/               # HotspotMap, ComplaintMarkers, HeatmapOverlay, HyderabadBoundaryLayer, canvasLayer, HotspotFilters/Legend/Stats
│  └─ ui/                 # Badge, Button, Card, Field, Logo, Modal, Pagination, StatCard, States, Timeline
├─ pages/
│  ├─ (public) Home, Login, Register, ForgotPassword, TrackComplaint, PublicEvents/Articles/Ngos/Lakes, Infrastructure
│  ├─ dashboard/ CitizenLayout, DashboardHome, ReportIssue, MyComplaints, ComplaintDetailPage, Notifications, Profile
│  ├─ officer/ FieldOfficerLayout, AssignedComplaints, FieldOfficerComplaintDetail, Notifications, Profile
│  └─ admin/ AdminLayout, AdminDashboard, ManageComplaints, Analytics, HotspotMapPage, Settings, AdminProfile,
│             community/ (Articles, Events, Lakes, Ngos, Pipelines, Plants)
```

16.3 Routing, state & auth

- **Route guards** (`RouteGuards.tsx`): `RequireAuth`, `RequireAdmin`, `RequireFieldOfficer`, `RedirectIfAuthenticated` — unauthenticated users are redirected to `/login` with the originating path; role mismatches redirect to the user's home path (`homePathFor(role)`).
- **Auth state** (`lib/auth.tsx`): React context. On boot the user is restored from `localStorage` (`sa_token`, `sa_user`). `login()` persists token + user; `refreshProfile()` re-fetches `GET /api/v1/auth/profile` for a fresh token; a global `sa:auth-expired` event (dispatched by the API client on any 401) clears the in-memory session so guards redirect immediately.
- **JWT storage/handling** (`lib/api.ts`): token in `localStorage` under `sa_token`; every request attaches `Authorization: Bearer <jwt>` and `X-Auth-User-Id: <id>`; `FormData` bodies skip the `Content-Type` header so the browser sets the multipart boundary.
- **API client**: typed `request<T>()` unwraps the `ApiResponse<T>` envelope, surfaces `ApiError` with HTTP status + application error code (e.g. `EMAIL_NOT_VERIFIED`), and dispatches `sa:auth-expired` on 401.
- **Forms/validation**: shared `Field` UI + client-side checks; backend Bean Validation is the source of truth (400 → field errors shown from the error envelope).
- **Toasts**: `lib/toast.tsx` global toast system for success/error feedback.

16.4 Role-based areas & capabilities

Area	Routes	Key functionality
Public	<code>/</code> , <code>/login</code> , <code>/register</code> , <code>/forgot-password</code> , <code>/track[:id]</code> , <code>/events</code> , <code>/articles</code> , <code>/ngos</code> , <code>/lakes</code> , <code>/infrastructure</code>	Landing, auth, complaint tracking by id, community content, NGO search, lake/infrastructure browsing
Citizen dashboard	<code>/dashboard/*</code>	Report issue (multipart upload), my complaints, complaint detail with timeline, notifications feed (polling), profile
Field officer	<code>/officer/*</code>	Assigned complaints, update status, resolve with proof photo, notifications, profile
Admin	<code>/admin/*</code>	Dashboard, manage complaints (assign via <code>AssignOfficerModal</code> , resolve with proof), analytics (Recharts), hotspot map (Leaflet heatmap + stats), community resource manager (articles/events/lakes/ngos/pipelines/plants), settings, profile

16.5 Frontend ↔ backend request flows



Figure 16.1 — The browser only ever talks to the frontend origin; all API traffic is proxied (Vite in dev, Vercel rewrite in prod), so CORS is a non-issue in the browser.

Example: citizen reports an issue — form → `POST /api/v1/complaints` (multipart) → gateway → complaint-service → Cloudinary + MySQL → `ComplaintResponse` → toast + redirect to detail; the RabbitMQ event later surfaces as an in-app notification polled by `useNotifications`.

17. Configuration & Environment Management

No real secrets in this document. All credential values below are shown as placeholders (<...>). Dev-only defaults in `application.yml` are *not* production credentials; compose files read the same values from environment variables.

17.1 Backend configuration model

- Each service has `src/main/resources/application.yml` with: server port, `spring.application.name` (registered with Eureka), datasource, JPA, RabbitMQ, Eureka client, service-specific `app.*/community.*/cloudinary.*/emailjs.*` blocks, and per-service logging levels.
- Spring Boot relaxed binding: env vars like `SPRING_DATASOURCE_URL`, `RABBITMQ_HOST`, `EUREKA_CLIENT_SERVICEURL_DEFAULTZONE` override YAML without code changes.
- Secrets/defaults use `${VAR:default}` syntax — e.g. `password: ${RABBITMQ_PASSWORD:guest}`; production compose switches to mandatory `${VAR:?...}` so missing env fails fast.

17.2 Environment variable inventory (all actual)

Variable	Used by	Purpose	Production value
RABBITMQ_HOST / PORT / USERNAME / PASSWORD	auth, complaint, notification	Broker connection	rabbitmq (compose) / <RABBITMQ_PASSWORD>
SPRING_DATASOURCE_URL / USERNAME / PASSWORD	all services	MySQL connection	jdbc:mysql://mysql:3306/<schema>... / <DATABASE_PASSWORD>
EUREKA_CLIENT_SERVICEURL_DEFAULTZONE	all services	Discovery registry	http://eureka-server:8761/eureka/
EUREKA_INSTANCE_PREFER_IP_ADDRESS	compose	Register container IP	"true"
CLOUDINARY_CLOUD_NAME / API_KEY / API_SECRET	complaint	Image uploads	<CLOUDINARY_*>
GNEWS_API_KEY	community	Article feed	<GNEWS_API_KEY>
GOOGLE_PLACES_API_KEY	community	NGO search	<GOOGLE_PLACES_API_KEY>
EMAILJS_SERVICE_ID / TEMPLATE_ID / PUBLIC_KEY / PRIVATE_KEY / WELCOME_TEMPLATE_ID	notification	Transactional email	<EMAILJS_*>
FRONTEND_URL	notification	Email links (login_url)	https://<frontend-domain>
VERIFICATION_TOKEN_TTL_MINUTES	auth	OTP lifetime	default 30
MYSQL_ROOT_PASSWORD	compose (mysql)	MySQL root password	<DATABASE_PASSWORD>
GATEWAY_PORT	compose prod	Host port for gateway	default 8080
VITE_API_URL / VITE_PROXY_TARGET	frontend	API base / dev proxy target	leave unset in prod (Vercel rewrite)

17.3 JWT configuration (auth-service)

```
app:
  jwt:
    secret: <JWT_SECRET>      # base64 dev key in source; use a strong secret in production
    expiration-ms: 86400000    # 24 hours
```

The committed default secret is a development-only value. Production must override it via environment/secret management; rotate it if the repository history ever exposed a real key.

17.4 Profiles & environment split

Environment	Datasource	Configuration source
Local (no compose)	H2 in-memory (auth/user/community), local MySQL (complaint/notification)	<code>application.yml</code> defaults; no env vars required
Docker dev	MySQL in container (<code>mysql:3306</code>)	<code>docker-compose.yml</code> env overrides; optional API keys
Production	MySQL in container	<code>docker-compose.prod.yml</code> + deployment <code>.env</code> (all secrets required)

There is a `application-mysql.yml` profile in auth-service for explicit MySQL use in local development (H2 remains the default).

18. Error Handling & Observability

18.1 Global exception handling (per service)

Every service has a `GlobalExceptionHandler` (`@RestControllerAdvice`) that converts exceptions into the uniform `ApiResponse` error envelope with appropriate HTTP statuses:

Exception (actual)	HTTP	Where
<code>EmailAlreadyExistsException</code>	409	auth
<code>InvalidCredentialsException</code>	401	auth
<code>ComplaintNotFoundException</code>	404	complaint
<code>ResolutionProofRequiredException</code>	400	complaint (v1.1.0)
<code>UserProfileNotFoundException</code>	404	user
<code>MethodArgumentNotValidException</code> (Bean Validation)	400 + field <code>errors</code> map	all
<code>AccessDeniedException</code>	403	auth (Security)
<code>Generic Exception</code>	500	all (logged)

```
// Error envelope shape (GlobalExceptionHandler)
{ "success": false, "message": "Complaint not found with id 42",
  "errors": { "title": "Title is required" } } // errors map present for validation failures
```

18.2 Failure modes by layer

Layer / failure	Behaviour (verified in code)
RabbitMQ producer failure	Logged, never thrown — API call already committed; event is lost (known trade-off, noted in Appendix A.6)
RabbitMQ consumer failure	5× retry with exponential backoff → reject without requeue → <code>notification.dlq</code> → DLQ consumer logs eventId for replay
Database failure	<code>@Transactional</code> rollback; complaint image flow compensates by deleting uploaded Cloudinary files via rollback synchronizations
File upload failure	Invalid type/size → 400 before upload; Cloudinary failure → 500 with logged error; proof upload failure → complaint stays unresolved
Email failure	Logged with eventId, never rethrown; notification still persisted; resend endpoint available
Feign / downstream failure	Role/existence checks surface as 4xx (handled in <code>GlobalExceptionHandler</code>); auth publishes are best-effort
Authentication failure	401 from auth-service; frontend dispatches <code>sa:auth-expired</code> and redirects to login

18.3 Logging & tracing reality

- **Logging:** SLF4J throughout; per-service `logging.level.com.sewagealert.<svc>: DEBUG` in dev; JPA `show-sql: true`; Eureka/Netflix discovery noise turned off in gateway/eureka.
- **No tracing stack configured** — there is no Micrometer tracing, Zipkin, or distributed trace IDs in the codebase. Cross-service correlation today relies on the `eventId` in notification events and standard log timestamps. A developer can trace a complaint event by: API log (service) → `eventId` in producer log → `eventId` in consumer/DLQ logs → notification row.
- **Monitoring tools:** none beyond RabbitMQ's management UI ([:15672](#)), Eureka dashboard ([:8761](#)), and H2/MySQL consoles. No Prometheus/Grafana/CloudWatch configuration exists.

19. Complete System Flow

This section ties together every verified component into one end-to-end picture.

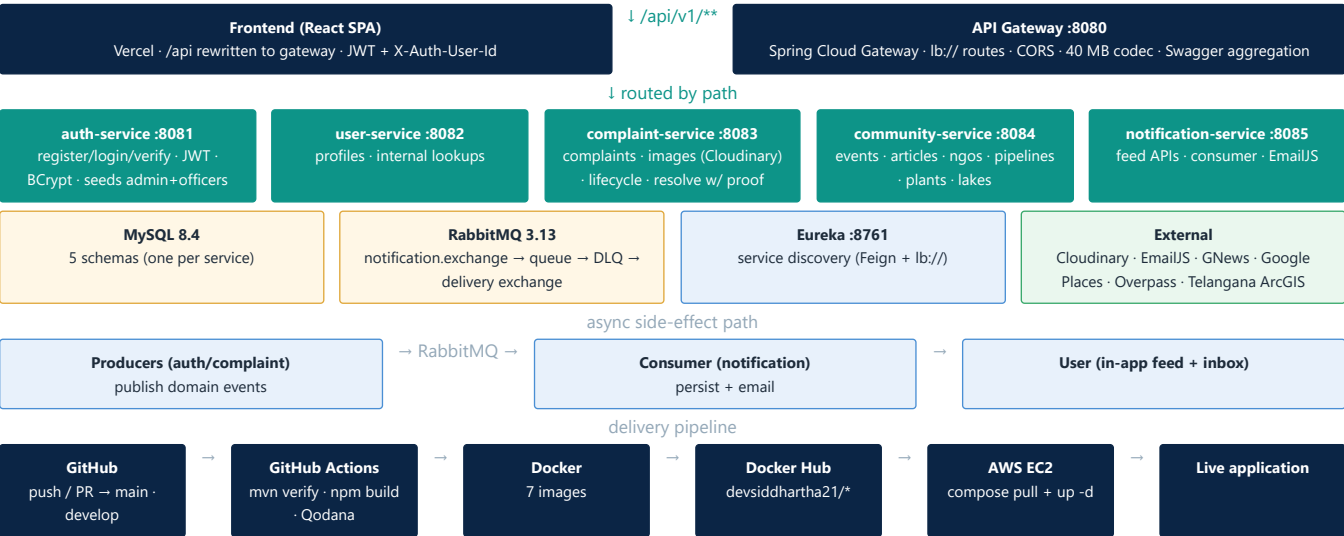


Figure 19.1 — Complete end-to-end system flow (frontend → gateway → services → data/messaging → external SaaS, plus the CI/CD delivery path).

19.1 The two communication planes

Plane	Mechanism	Purpose	Examples
Synchronous request/response	REST via gateway + Feign/Eureka internally	User-facing operations and decisions	Login, create complaint, assign, notifications feed
Asynchronous events	RabbitMQ topic exchange	Side effects after committed business actions	Complaint created/assigned/resolved → notification + email
Delivery pipeline (CI/CD)	GitHub Actions → Docker Hub → EC2	Ship code to production	docker-build-push on main

20. Developer Guide

20.1 Repository structure

```
sewage-alert-hyderabad/
├─ pom.xml                # Maven multi-module reactor (Java 25)
├─ api-gateway/ eureka-server/ auth-service/ user-service/
├─ complaint-service/ community-service/ notification-service/
├─ frontend/              # React 18 + TS + Vite
├─ docker/                # mysql init SQL (creates the 5 databases)
├─ docker-compose.yml     # full dev stack (builds locally)
├─ docker-compose.prod.yml # production stack (pulls images)
├─ Dockerfile             # whole-reactor image (SERVICE_NAME runtime switch)
├─ .github/workflows/     # backend-ci, frontend-ci, docker-build-push, qodana
└─ docs/                  # design docs, meeting notes, presentations
```

20.2 Prerequisites

- JDK 25 (Temurin) + Maven 3.9+ + Node.js 20 + npm
- Docker Desktop (for RabbitMQ, and for compose-based runs)
- MySQL 8 (optional in dev — auth/user/community default to H2; complaint & notification need MySQL, or use Docker)
- API keys (optional for local dev): `CLOUDINARY_*` (uploads), `EMAILJS_*` (emails), `GNEWS_API_KEY`, `GOOGLE_PLACES_API_KEY`

20.3 Fastest path — Docker Compose (full stack)

```
# from the repository root
docker compose up -d --build

# Eureka dashboard    http://localhost:8761
# API gateway         http://localhost:8080
# RabbitMQ console    http://localhost:15672 (guest/guest in dev)
# MySQL               localhost:3307       (root / dev password from .env)
```

20.4 Manual backend startup (per service)

```
# 0. message broker (RabbitMQ)
cd notification-service && docker compose up -d # or any local broker on :5672

# 1. discovery server
cd eureka-server && mvn spring-boot:run

# 2. business services (own terminals)
cd auth-service && mvn spring-boot:run          # :8081
cd user-service && mvn spring-boot:run          # :8082
cd complaint-service && mvn spring-boot:run      # :8083 (needs MySQL schema sewagealert_complaints)
cd community-service && mvn spring-boot:run      # :8084
cd notification-service && mvn spring-boot:run   # :8085 (needs MySQL schema sewagealert_notifications)

# 3. API gateway (last)
cd api-gateway && mvn spring-boot:run          # :8080
```

For complaint/notification services create the databases first: `CREATE DATABASE sewagealert_complaints; / CREATE DATABASE sewagealert_notifications;` (or use the compose MySQL which runs `docker/mysql/init/01-create-databases.sql`). Set `CLOUDINARY_*` before starting complaint-service or image uploads will fail at runtime.

20.5 Frontend startup

```
cd frontend
npm install
npm run dev # http://localhost:5173 — Vite proxies /api → http://localhost:8080
```

20.6 Swagger & API testing

- Aggregated UI: <http://localhost:8080/swagger-ui.html> (gateway) or per-service [:8081-8085/swagger-ui.html](http://localhost:8081-8085/swagger-ui.html).
- Quick smoke test with curl:

```
curl -s http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@sewagealert.com","password":"Admin@123"}' # dev seed (ADMIN)
```

20.7 Tests, builds & Docker images

```
# backend: build + run all tests (reactor)
mvn --batch-mode clean verify

# note: one notification-service integration test requires a Docker daemon

# frontend: lint + test + build
cd frontend && npm run lint --if-present && npm test --if-present && npm run build

# single service image (two-stage maven build)
docker build -f auth-service/Dockerfile -t sewagealert/auth-service .
```

20.8 Deployment overview

1. Push to `main` → GitHub Actions runs `mvn clean verify` and pushes 7 images (`latest` + SHA) to Docker Hub.
2. On the EC2 host, prepare a deployment `.env` (all `${VAR:?}` values required).
3. `docker compose -f docker-compose.prod.yml --env-file .env pull` then `... up -d`.
4. Frontend: Vercel build reads `frontend/`, rewrites `/api/*` to the gateway (address configured in `vercel.json`), and serves the SPA.

21. Troubleshooting

Issues below are grounded in the actual architecture and configurations verified in this repository.

Symptom	Likely cause	Fix
Service fails to start; "Connection refused" for MySQL	MySQL not running or wrong credentials; <code>ddl-auto</code> needs an existing schema	Create the schema (<code>CREATE DATABASE sewagealert_<svc></code>), check <code>SPRING_DATASOURCE_*</code> env vs <code>application.yml</code> ; use compose MySQL on <code>localhost:3307</code>
"Connection refused" for RabbitMQ	Broker down or wrong host/credentials	<code>cd notification-service && docker compose up -d</code> ; verify <code>RABBITMQ_HOST/PORT/USERNAME/PASSWORD</code> ; check UI at :15672
Registration succeeds but no verification email arrives	<code>EMAILJS_*</code> not set; EmailJS template params mismatch	Set env vars; confirm template uses <code>{{name}}/{{email}}/{{verification_code}}</code> ; producer publishes are fire-and-forget — check notification-service logs for the eventId
Login returns 401 EMAIL_NOT_VERIFIED	Account not verified	Use <code>POST /api/v1/auth/resend-verification</code> (throttled 1/min), then verify the OTP
401 / session lost on every call	Expired JWT (24 h), clock skew, or wrong secret	Re-login; ensure all services use the same <code>app.jwt.secret</code> ; frontend auto-clears on <code>sa:auth-expired</code>
CORS errors in browser	Calling the gateway directly from the SPA origin instead of via proxy/rewrite	Dev: use Vite proxy (same-origin); prod: keep <code>VITE_API_URL</code> unset so Vercel rewrites <code>/api</code> ; gateway CORS is already permissive
Gateway returns 503 / "Unable to find instance"	Service not registered with Eureka	Start eureka first, then services; check Eureka dashboard :8761; in containers verify <code>EUREKA_CLIENT_SERVICEURL_DEFAULTZONE</code> and <code>EUREKA_INSTANCE_PREFER_IP_ADDRESS=true</code>
Image upload fails at runtime	Cloudinary credentials missing	Set <code>CLOUDINARY_CLOUD_NAME/API_KEY/API_SECRET</code> ; without them the service starts but uploads fail (by design)
Complaint cannot be marked RESOLVED (400 ResolutionProofRequired)	Using the JSON PATCH status endpoint	Use the multipart resolve endpoints (<code>.../resolve</code>) with a proof image — PATCH deliberately rejects RESOLVED
Multipart upload rejected at gateway	Request exceeds codec/multipart limits	Per-file ≤ 10 MB, request ≤ 40 MB (limits in gateway + complaint-service); JPG/PNG/WEBP only
Frontend build fails on Vercel / deep route 404	Missing SPA fallback or committed artifacts	<code>vercel.json</code> rewrites <code>/(.*)/index.html</code> ; never commit <code>node_modules</code>
Backend build fails	Wrong JDK or missing modules	Use Temurin JDK 25 + Maven 3.9; run from the reactor root (<code>mvn clean verify</code>); auth-service needs user-service sources (handled by <code>-am</code>)
Events not reaching notifications	Routing-key mismatch; consumer retries exhausted → DLQ	Binding is <code>notification.#</code> (covers 2- and 3-word keys); check <code>notification.dlq</code> for failed eventIds via <code>DeadLetterQueueConsumer</code> logs

22. Technical Glossary

Term	Meaning (in this project)
REST API	HTTP-based JSON API served by each microservice; accessed through the gateway under <code>/api/v1</code>
DTO	Data Transfer Object — request/response payload classes (e.g. <code>RegisterRequest</code> , <code>ComplaintResponse</code>) that decouple the API from JPA entities
JPA / Hibernate	Java Persistence API + its Hibernate implementation; entities mapped to MySQL/H2 tables with <code>ddl-auto: update</code>
JWT	JSON Web Token — HS256-signed token (claims: <code>sub</code> =userId, email, role) issued by auth-service, valid 24 h
Filter	<code>JwtAuthenticationFilter</code> — Spring Security <code>OncePerRequestFilter</code> that validates the Bearer token and populates the <code>SecurityContextHolder</code>
SecurityContext	Thread-local Spring Security store holding the authenticated principal (userId + <code>ROLE_<role></code> authority) for the current request
RabbitMQ	AMQP message broker used for async domain events; container <code>rabbitmq:3.13-management</code> (AMQP :5672, UI :15672)
Exchange	<code>notification.exchange</code> — durable topic exchange receiving all published events
Queue	<code>notification.queue</code> — durable queue bound with <code>notification.#</code> ; consumed by <code>NotificationEventConsumer</code>
Routing key	Topic key on each message (e.g. <code>notification.user.registered</code>) matched against bindings to route to queues
DLQ / DLX	Dead-letter queue/exchange — <code>notification.dlq/notification.dlx</code> hold messages rejected after retry exhaustion
Feign	OpenFeign declarative HTTP client used for synchronous service-to-service calls (Eureka-resolved)
Eureka	Netflix service registry (:8761); services register by name, gateway routes <code>lb://SERVICE-NAME</code>
API Gateway	Spring Cloud Gateway (:8080) — single entry point, path-based routing, CORS, Swagger aggregation
Microservice	One of the 5 business services (+ gateway + eureka), each with its own DB, port, and deployable unit
Docker / Container	Image-based packaging; two-stage Maven/Temurin images; compose wires all containers on one bridge network
CI/CD	GitHub Actions: backend/frontend CI + Docker build & push to Docker Hub; manual deploy step on EC2
AWS	Cloud provider; in this project the EC2 Docker host for the backend + (implicit) Vercel for the frontend — no AWS-native services configured
Swagger / OpenAPI	springdoc-generated interactive API docs (<code>/swagger-ui.html</code> , <code>/v3/api-docs</code>) aggregated at the gateway
Cloudinary	External image storage/CDN holding complaint photos and mandatory resolution-proof images
EmailJS	External email API used server-side for verification + welcome emails
Bean Validation	Jakarta validation annotations (<code>@NotBlank</code> , <code>@Email</code> , <code>@Size</code> ...) on DTOs enforced by <code>@Valid</code> at controllers
ApiResponse<T>	Uniform response envelope: <code>success</code> , <code>message</code> , <code>data</code> , optional <code>errors</code>

A. Appendices — Inventories & Summaries

A.1 Complete technology stack

Layer	Technology	Version / notes
Language	Java (backend) · TypeScript (frontend)	Java 25 · TS with React 18
Backend framework	Spring Boot	3.5 (Web MVC + WebFlux for gateway)
Microservices stack	Spring Cloud Gateway · Netflix Eureka · OpenFeign	Gateway :8080 · Eureka :8761
Security	Spring Security + jwt	HS256, 24 h expiry, BCrypt passwords
Messaging	RabbitMQ (Spring AMQP)	3.13; topic exchange + DLQ + retries
Databases	MySQL · H2 (dev)	MySQL 8.4 in containers; 5 schemas
Persistence	Spring Data JPA / Hibernate	ddl-auto update, derived queries
API docs	springdoc-openapi	Swagger UI + aggregated gateway docs
Frontend	React · Vite · React Router 6 · Leaflet · Recharts	Node 20; manual chunking
Image storage	Cloudinary	folder "complaints", secure_url stored
Email	EmailJS REST API	server-side, verification + welcome templates
External data	GNews · Google Places · Overpass (OSM) · Telangana ArcGIS	Feign clients with per-client timeouts
Build / run	Maven · npm · Docker	Multi-module reactor; two-stage images; compose dev + prod
CI/CD	GitHub Actions · Docker Hub	4 workflows; SHA + latest tags
Hosting	AWS EC2 (backend) · Vercel (frontend)	compose.prod on EC2; vercel.json rewrites

A.2 Complete service / module inventory

Service	Port	Database	Key packages	Responsibilities
eureka-server	8761	—	—	Service discovery registry (self-preservation off)
api-gateway	8080	—	config (OpenApiConfig)	Routing, CORS, codec sizing, Swagger aggregation
auth-service	8081	sewagealert_auth	controller, security, service.impl, model, dto, producer, client, config, exception, util	Registration, login, OTP verification, JWT, seeders, publish auth events
user-service	8082	sewagealert_users	controller, service.impl, model, dto	User profiles CRUD + internal lookups
complaint-service	8083	sewagealert_complaints	controller, service.impl, model, dto, storage, client, producer, config, repository	Complaint lifecycle (create→assign→status→resolve w/ proof), Cloudinary uploads, publish events
community-service	8084	sewagealert_community	controller, service.impl, model, client, config	Events + registrations, articles (+GNews), NGOs (+Google Places/Overpass), pipelines, treatment plants (+ArcGIS), lakes (+Overpass)
notification-service	8085	sewagealert_notifications	controller, consumer, service.impl, model, dto, client, producer, config, repository, util, enums	Consume events → persist notifications + email via EmailJS; feed APIs; DLQ consumer; delivery re-publish

A.3 Complete API inventory

Every endpoint documented in §4 — summary counts: auth 8 (6 public/1 admin + 1 internal), user 7 (+1 internal), complaint 10 (citizen 5 + admin 2 + officer 3), notification 5, community 30 (6 groups × 5 + extras: events register/upcoming, articles latest, ngos search, plants/lakes external). All under [/api/v1](#).

A.4 Complete infrastructure inventory

Infrastructure	Detail
Networks	Single bridge network backend (compose); Vercel rewrites for frontend→gateway
Volumes	mysql-data, rabbitmq-data
Ports	8080 gateway (only public in prod) · 8761 eureka · 8081–8085 services · 5672/15672 rabbitmq · 3306/3307 mysql · 5173 vite
Health checks	MySQL mysqladmin ping; RabbitMQ rabbitmq-diagnostics ping; compose depends_on conditions
Registries	Docker Hub (devsiddhartha21/<svc>:latest <sha>)
External services	Cloudinary, EmailJS, GNews, Google Places, Overpass, Telangana ArcGIS
CI/CD	GitHub Actions (backend-ci, frontend-ci, docker-build-push, Qodana) + GitHub secrets/vars

A.5 Request / event flow summary

Flow	Path	Sync / async
Register + verify	Citizen → gateway → auth (BCrypt, save) → Feign user profile → publish USER_REGISTERED → notification consumer → EmailJS + in-app	Sync (API) + async (email)
Login	Citizen → gateway → auth (DAO check, JWT) → frontend stores sa_token/sa_user	Sync
Authenticated call	Frontend (Bearer + X-Auth-User-Id) → gateway → service (auth: JWT filter; others: header + Feign role re-check)	Sync
Create complaint	Citizen → gateway → complaint (Feign user check, validate, Cloudinary upload, save, publish COMPLAINT_CREATED)	Sync + async notification
Assign complaint	Admin → gateway → complaint (Feign role ADMIN + officer role, save, publish ASSIGNED → officer notified)	Sync + async notification
Resolve complaint	Admin/officer → gateway → complaint (role + ownership, mandatory proof upload to Cloudinary, save RESOLVED, publish RESOLVED → citizen notified)	Sync + async notification
Notifications	Producer → RabbitMQ → consumer (persist + optional email) → feed APIs; delivery exchange ready for future channels	Async

A.6 Known limitations & missing documentation (found during inspection)

- **No automated deployment step** — docker-build-push stops at the registry; EC2 pull/up is manual (no CD job).
- **No distributed tracing** — no Micrometer/Zipkin; cross-service correlation relies on `eventId` and logs.
- **Producer side-effect loss** — RabbitMQ publish failures are logged and dropped (fire-and-forget), so a broker outage can lose a notification (consumer side has retries + DLQ).
- **JWT validated only in auth-service** — other services trust `X-Auth-User-Id` + Feign role re-checks; a gateway-level JWT filter would harden the trust boundary.
- **Dev defaults in source** — H2/in-memory DBs, `guest/guest` broker, seeded admin/officer passwords, and a dev JWT secret in `application.yml`; production must override via env.
- **Community service external APIs** — GNews/Google Places require keys; without them those features degrade or fall back (Overpass).
- **No automated end-to-end tests** — CI runs unit/integration tests in the Maven reactor (one notification test needs Docker) and frontend build checks; no full-stack E2E suite.
- **Hardcoded EC2 address in vercel.json** — the frontend rewrite target is a committed IP; it must be updated when the host changes (kept as placeholder in this document).

A.7 Planned / future features (marked separately)

- **Future:** bound consumers on `notification.delivery.exchange` (email/SMS/push workers) — topology and re-publish logic are already in place.
- **Future:** `forgot-password` route exists in the frontend route table but no backend password-reset flow was found (login/verify only).
- **Future:** production JWT secret management, AWS-native services (RDS/S3/CloudWatch), and a CD job — none present in the repository.

End of document — Sewage Alert Hyderabad - Technical Architecture & Implementation Guide - generated from repository inspection, August 2026.